Week 14 easier

Week 14: Deep Learning for Audio. Treating STFT spectrograms as images. Building Convolutional Neural Networks (CNNs) in PyTorch to classify urban sounds. Overfitting, dropout, and validation.

Week 14: Deep Learning for Audio. Treating STFT spectrograms as images. Building Convolutional Neural Networks (CNNs) in PyTorch to classify urban sounds. Overfitting, dropout, and validation.

Please map out two 75 min lectures for week 14 (lectures 27 and 28). The students have seen an introduction to PyTorch in week 13 (lectures 25 and 26). The class is a senior/graduate level DSP and ML course. The textbooks are Oppenheim/Schafer "Discrete-Time Signal Processing" and Raschka "Machine Learning with PyTorch and Scikit-Learn"

Gemini Pro:

Here is the lecture roadmap for Week 14, mapping the transition from classical DSP representations to deep learning architectures. This plan bridges Oppenheim and Schafer’s time-frequency analysis with Raschka’s PyTorch implementations.

Lecture 27: From DSP to Deep Learning – Audio as Images

Focus: Bridging classical time-frequency analysis (STFT) to neural network inputs, handling dataset variability, and building the PyTorch data pipeline.

Time	Topic	Key Concepts & Teaching Points	Readings / Resources
0-15m	**Recap: Time-Frequency Analysis**	Review STFT and Welch's method. Discuss the transition from linear frequency scales to the Mel scale to capture perceptually relevant features.	O&S: Ch. 10 (Fourier Analysis of Signals)
15-30m	**Spectrograms as Images**	Explain why audio classification can be treated as a computer vision	Raschka: Ch. 14 (CNNs)

Time	Topic	Key Concepts & Teaching Points	Readings / Resources
		problem. Define the 2D matrix: Frequency (Mel bins) × Time (Frames) as a 1-channel grayscale image.	
30-50m	**The PyTorch Audio Pipeline**	Walk through building a custom PyTorch `Dataset`. Show how to use `torchaudio` or `librosa` to load audio, standardize sample rates, and extract Mel-spectrograms on the fly.	
50-65m	**Patching & Dimensionality**	Address variable-length audio. Teach techniques for slicing continuous audio into fixed-length patches (e.g., 3-second windows) and zero-padding short clips to ensure consistent `[Batch, 1, n_mels, Time]` tensor shapes.	
65-75m	**Scaling & Normalization**	Explain the necessity of converting amplitude squared (power) to decibels (dB). Show how standardizing patches (zero mean, unit variance) stabilizes gradient descent.	

Lecture 28: CNN Architectures, Training, and Regularization

Focus: Designing lightweight 2D CNNs for spectrograms, executing the training loop, and preventing overfitting on datasets like UrbanSound8K or ESC-50.

Time	Topic	Key Concepts & Teaching Points	Readings / Resources
0-25m	**CNN Architectures for Audio**	Map Conv2d layers to audio features (capturing local spectro-temporal patterns). Introduce `nn.AdaptiveAvgPool2d` as a strategy to handle variable time dimensions without breaking the fully connected classification head.	Raschka: Ch. 14 & 15
25-45m	**The Training Loop**	Construct the full PyTorch training step. Connect the output logits to `nn.CrossEntropyLoss`. Discuss optimizer selection (AdamW vs. SGD) and learning rate scheduling (e.g., Cosine Annealing) for audio tasks.	Raschka: Ch. 12 (Training NNs)
45-60m	**Overfitting & Dropout**	Analyze the training vs. validation loss curves. Demonstrate how adding `nn.Dropout` before the linear classifier prevents feature co-adaptation, especially on smaller environmental audio datasets.	Raschka: Ch. 13 (Regularization)

Time	Topic	Key Concepts & Teaching Points	Readings / Resources
60-75m	**Domain-Specific Augmentation**	Introduce SpecAugment. Explain why masking blocks of frequency channels and time steps directly on the spectrogram is highly effective for audio, acting as a specialized form of dropout.	

> **Instructor Note:** When discussing the CNN architecture in Lecture 28, it is highly effective to contrast the deep learning approach with classical DSP feature extraction (like tracking specific harmonic peaks or using filter banks directly). Highlight that the CNN's first convolutional layers are essentially learning optimized, task-specific FIR filter banks from the data.

Prompt: Can you create slides and python code for these two 75 minute classes of a DSP and ML course for a combined senior and first year Electrical and Computer Engineering class? The DSP textbook is Discrete-Time Signal Processing by Oppenheim and Schafer. The ML textbook is Machine Learning with PyTorch and Scikit-Learn by Raschka. Add some homework problems with solutions and a weekly quiz with solutions as well.

ChatGPT: (Medium)

Week 14, Lecture 27: From DSP to Deep Learning – Audio as Images

Focus: Bridging classical time-frequency analysis (STFT) to neural network inputs, handling dataset variability, and building the PyTorch data pipeline.

Time	Topic	Key Concepts & Teaching Points	Readings / Resources
0-15m	**Recap: Time-Frequency Analysis**	Review STFT and Welch's method. Discuss the transition from linear frequency scales to the Mel scale to	O&S: Ch. 10 (Fourier Analysis of Signals)

Time	Topic	Key Concepts & Teaching Points	Readings / Resources
		capture perceptually relevant features.	
15-30m	**Spectrograms as Images**	Explain why audio classification can be treated as a computer vision problem. Define the 2D matrix: Frequency (Mel bins) × Time (Frames) as a 1-channel grayscale image.	Raschka: Ch. 14 (CNNs)
30-50m	**The PyTorch Audio Pipeline**	Walk through building a custom PyTorch `Dataset`. Show how to use `torchaudio` or `librosa` to load audio, standardize sample rates, and extract Mel-spectrograms on the fly.	
50-65m	**Patching & Dimensionality**	Address variable-length audio. Teach techniques for slicing continuous audio into fixed-length patches (e.g., 3-second windows) and zero-padding short clips to ensure consistent `[Batch, 1, n_mels, Time]` tensor shapes.	
65-75m	**Scaling & Normalization**	Explain the necessity of converting amplitude squared (power) to decibels (dB). Show how standardizing	

Time	Topic	Key Concepts & Teaching Points	Readings / Resources
		patches (zero mean, unit variance) stabilizes gradient descent.	

Prompt: Can you create slides and python code for this 75 minute class of a DSP and ML course for a combined senior and first year Electrical and Computer Engineering class? The DSP textbook is Discrete-Time Signal Processing by Oppenheim and Schafer. The ML textbook is Machine Learning with PyTorch and Scikit-Learn by Raschka. Add some homework problems with solutions and a weekly quiz with solutions as well.

ChatGPT-5.6 Sol Medium: (worked for 20 min, 22 sec)

Created the complete Week 14 package:

- PowerPoint slides — 38 slides
- Companion Jupyter notebook

The deck includes the 75-minute lecture, teaching prompts, in-class exercises, six homework problems with solutions, and an eight-question weekly quiz with solutions.

The notebook includes synthetic audio demonstrations, STFT and Mel-spectrogram generation, cropping and padding, dB conversion, normalization, custom PyTorch datasets, batching, and a small CNN interface test. It follows the current TorchAudio feature-extraction APIs. PyTorch was unavailable in the creation environment, so the notebook received syntax validation but not full numerical execution.

Week 14, Lecture 28: CNN Architectures, Training, and Regularization

Focus: Designing lightweight 2D CNNs for spectrograms, executing the training loop, and preventing overfitting on datasets like UrbanSound8K or ESC-50.

Time	Topic	Key Concepts & Teaching Points	Readings / Resources
0-25m	**CNN Architectures for Audio**	Map Conv2d layers to audio features (capturing local spectro-temporal patterns). Introduce `nn.AdaptiveAvgPool2d` as a strategy to	Raschka: Ch. 14 & 15

Time	Topic	Key Concepts & Teaching Points	Readings / Resources
		handle variable time dimensions without breaking the fully connected classification head.	
25-45m	**The Training Loop**	Construct the full PyTorch training step. Connect the output logits to `nn.CrossEntropyLoss`. Discuss optimizer selection (AdamW vs. SGD) and learning rate scheduling (e.g., Cosine Annealing) for audio tasks.	Raschka: Ch. 12 (Training NNs)
45-60m	**Overfitting & Dropout**	Analyze the training vs. validation loss curves. Demonstrate how adding `nn.Dropout` before the linear classifier prevents feature co-adaptation, especially on smaller environmental audio datasets.	Raschka: Ch. 13 (Regularization)
60-75m	**Domain-Specific Augmentation**	Introduce SpecAugment. Explain why masking blocks of frequency channels and time steps directly on the spectrogram is highly effective for audio, acting as a specialized form of dropout.	

Prompt: Can you create slides and python code for this 75 minute class of a DSP and ML course for a combined senior and first year Electrical and Computer Engineering class? The DSP textbook is Discrete-Time Signal Processing by Oppenheim and Schafer. The ML textbook is Machine Learning with PyTorch and Scikit-Learn by Raschka. Add some homework problems with solutions and a weekly quiz with solutions as well.

ChatGPT-5.6 Sol Medium: (worked for 13 min, 25 sec)

Created the complete Lecture 28 package:

- Lecture 28 slides — 45-slide PowerPoint
- Lecture 28 Python notebook

Included:

- Lightweight spectrogram CNN architecture and tensor-shape audits
- `AdaptiveAvgPool2d` with variable-duration inputs
- Complete AdamW/Cosine Annealing training and validation loop
- Cross-entropy, logits, dropout, learning-curve diagnosis, and SpecAugment
- Self-contained synthetic spectrogram dataset requiring no external download
- UrbanSound8K and ESC-50 fold-protocol guidance
- In-class activities, six homework problems with worked solutions, and an eight-question weekly quiz with solutions
- Instructor notes and primary-source links throughout

The deck passed structural and layout validation with no findings. All 45 slides were rendered and visually inspected. The notebook's JSON and Python syntax were validated; it was not executed because PyTorch is unavailable in this runtime. Implementation follows the current PyTorch documentation and the official UrbanSound8K and ESC-50 dataset protocols.

Week 14, Lecture 27

Explain in more detail what's going on here and what good answers and/or examples are. For all mathematical equations, use standard single dollar signs . .. for inline math and double dollar signs

. ..

for block/display math. Do not use (...) or [...]. Output the raw text without escaping the dollar signs.

Chat GPT (Medium):

Gets stuck

ClaudeAI Sonnet 5.5 Medium:

Week 14, Lecture 27: Audio as Images

The big picture

The lecture's claim is that a model's input is the end of a chain of DSP and data-handling decisions, and each decision decides what the model can and can't learn. The chain is:

waveform → STFT → power → Mel filter bank → dB → normalization → fixed-size tensor `[B, 1, M, T]` → CNN

Each stage throws something away or fixes a convention:

Stage	What it does	What it discards or commits to
Resample / mono	Gives every cell the same physical meaning	Spatial info, if you average channels
STFT	Local spectra, so time order is kept	Frequency resolution vs. time resolution trade-off
Power `|X|²`	Local energy distribution	Phase
Mel bank	Pools bins, with coarser resolution at high frequency	Fine high-frequency detail
dB	Compresses dynamic range	Needs a floor, reference, and consistent convention
Normalization	Stabilizes training	Absolute level, if done per patch
Crop / pad	Makes batching possible	Anything outside the crop, plus a possible padding shortcut

The second theme is that most of the dangerous failures are silent. A leaky split or a wrongly squeezed tensor doesn't crash anything. It produces a number that looks fine, or an error far from the real cause. That is why the lecture stresses a "representation contract" at the Dataset boundary and why the quiz line says strong answers name both the failure and its mechanism.

Walkthrough of the sections

Slides 3–7: why STFT? A global spectrum (DFT, periodogram, or Welch) can't distinguish a rising chirp from a falling one because both contain the same frequencies. Welch averages away timing on purpose to reduce variance. The STFT keeps a spectrum per frame, so a chirp shows up as a diagonal ridge. The rule from slide 4 is to choose the representation whose invariances match the task.

The grid is set by two formulas. Bin spacing is $\Delta f = f_s/N$ and frame spacing is $\Delta t = R/f_s$.

- A longer window gives a narrower main lobe, so close tones separate better, but fast changes are localized worse.
- A smaller hop gives more frames and more overlap, but the information is more redundant and costs more compute.
- A larger `n_fft` gives more sampled bins, but if it is just zero padding, the physical resolution still depends on the window.

Slides 8–13: Mel and the image analogy. The Mel filter bank is a change of coordinates, not new information. It pools neighboring bins with wider pooling at high frequency, and it reduces dimension. The image analogy helps because local patterns carry meaning (ridges for tones, stripes for onsets, diagonals for chirps) and convolution shares detectors across the map. It also has limits: the axes have different units, shifting in time usually preserves the label while shifting in frequency often doesn't, and intensity is log energy rather than reflected light.

Slides 14–21: the Dataset as a contract. `Dataset.__getitem__` loads, converts to mono, resamples, crops or pads, computes the Mel spectrogram and dB, normalizes, and returns one consistent example. The DataLoader only stacks. The key points:

- Resample before feature extraction, or the same `n_fft` means different Hz and seconds.
- Use random crops for training (augmentation) and deterministic crops for validation and test, so the metric isn't noisy.
- Zero padding creates a shortcut if clip duration correlates with class, so keep `valid_samples` or a mask.

Slides 22–25: scale and shape. dB is `10·log₁₀(P/P_ref)`, so you need a floor to avoid −∞. Normalization choices encode invariances: per-patch removes absolute level, dataset-level preserves it, and per-bin compensates for spectral tilt. Statistics must be fit on training data only. Caching is fine once the representation has stabilized, and the cache key must include the parameters.

Slides 26–27: leakage and reproducibility. Split by source group (speaker, patient, session) before extracting patches. Otherwise patches from the same recording land in train and test, and the model memorizes the speaker, room, or device. Reproducibility means seeds, a fixed split manifest, deterministic eval transforms, logged parameters, and an inspected first batch.

Worked answers to the in-class items

Opening diagnostic (slide 3): The STFT or spectrogram separates the two recordings. The global spectrum has the same frequency content for both, but the STFT keeps which frequencies occur when. A strong answer says that A is a diagonal ridge going up and B is a diagonal ridge going down.

Activity step 1 (predict T):

$T = 1 + \lfloor (L - N)/R \rfloor = 1 + \lfloor (48{,}000 - 512)/160 \rfloor = 1 + \lfloor 296.8 \rfloor =$ **297**

The frame count is 1 + floor, not just floor. The floor means the last partial frame is dropped when `center=False`.

Activity step 3 (why `squeeze()` breaks Conv2d): Each item is `[1, 64, 297]`. Squeezing it gives `[64, 297]`, and after batching you get `[8, 64, 297]`. Conv2d expects `[B, C, H, W]`, so the channel axis is gone. Recent PyTorch may interpret the 3D tensor as an unbatched `[C, H, W]` and treat 8 as channels, which gives a confusing channel-mismatch error. If the batch size is 1, `squeeze()` also deletes the batch axis. The fix is to be explicit, with `squeeze(0)` or `unsqueeze(1)`, rather than an unconstrained `squeeze()`.

Activity step 4: Use a center crop, or fixed multi-crop windows, with no randomness in the transform.

Exit check (slide 30): The strongest answer is splitting after patch extraction instead of by group. It raises no error, and the metrics just look great. Other good answers are fitting normalization statistics on all data including test, and padding length that correlates with class. A weak answer names the choice without the mechanism.

Homework: what a strong answer looks like

1. **Frame count:** L = 55,125, bins = 513, frames = 212. Show the formula and the rounding, not just the numbers.
2. **7.2 kHz fault:** Start with a linear spectrogram or a dense high-frequency Mel bank. A standard Mel bank gives few bands up high and can blur a narrow line. A great answer adds a quick check, such as plotting both for a synthetic 7.2 kHz tone.
3. **Crop policy for 30 s clips with a 1 s event:** Random crops of 30 s would mostly miss the event, so the label may not apply to the crop. Use event timestamps for positive training crops, add labeled background crops, evaluate with fixed sliding windows, and aggregate with a stated rule (max for rare events), validated at the clip level.
4. **dB values:** 0, −6.02, −20, −50 dB. Clamp because $P = 0$ gives $-\infty$, which breaks the normalization and gradients.
5. **Shape bug:** The channel axis was removed, usually by `squeeze()`. Repair 1 is to keep each item `[1, 64, 297]` in the Dataset. Repair 2 is `x.unsqueeze(1)` at the batch level, while also fixing the Dataset.

6. **Respiratory leakage:** Split by patient, never by clip, and fit normalization on training patients only. The checks are an empty patient-ID intersection between splits, and a classifier trained to predict patient or recording source (if it does well, there's a collection signature to exploit). A near-duplicate hash check is another.

Quiz: model answers

1. Frame-level time localization, meaning the temporal order of spectral content.
2. More frames per second and more overlap. Frequency-bin spacing doesn't change.
3. It gives FFT bins, frame spacing, duration, and Mel limits the same physical meaning.
4. `[1, n_mels, frames]`, which becomes `[B, 1, n_mels, frames]` after batching.
5. −20 dB.
6. Each evaluation pass can score a different segment, so crop noise ends up in the metric.
7. The model can use padded duration or the padding boundary if it correlates with class.
8. The split was by clip instead of by patient group, so there is dependency leakage.

Things worth knowing beyond the slides

- **Padding vs. normalization:** Per-patch normalization (slide 28) computes the mean and standard deviation over zero-padded regions too, so padding changes the statistics. That's another reason to keep a validity mask.
- **Mel vs. linear:** Mel is a prior that low frequencies matter more, which suits speech and music. For narrowband machine faults at high frequency, that prior can hurt.
- **Phase:** Discarding phase is usually fine for classification. It matters if the label depends on timing at sub-frame resolution, or if you need to reconstruct audio.

If useful, I can turn this into a study guide, make practice problems at a different difficulty, or write the `MelPatchDataset` code with the validity-mask extension.

Explain in more detail what's going on here and what good answers and/or examples are.

Gemini Auto:

The slide deck provides a complete pedagogical and technical bridge between classical Digital Signal Processing (DSP) and modern 2D Convolutional Neural Network (CNN) pipelines for audio processing.

Below is an in-depth breakdown of what is happening across the core topics, along with high-mark academic and technical answers for the in-class diagnostic, the coding tasks, the weekly quiz, and the homework problems.

---

1. Deep Dive: What Is Happening in This Lecture?

This lecture addresses a core engineering challenge: **How do we format unstructured, variable-length 1D audio signals into deterministic, physically meaningful 2D tensors ready for GPU-accelerated CNNs without introducing data leakage or numeric instability?**

Key Modules Covered

1. **Time-Frequency Foundations (STFT vs. Welch):** Understanding why global spectra lose temporal order and how window length (N) vs. hop size (R) dictates the physical time-frequency grid.

2. **Audio-to-Image Mapping & Mel Filtering:** Mapping linear STFT bins to non-linear Mel filterbanks. It highlights both the strength of 2D CNN spatial inductive biases (local feature reuse for audio harmonics, onsets, and chirps) and its failure modes (frequency translation invariance does *not* imply semantic invariance in audio like translation does in computer vision).

3. **PyTorch Tensor Engineering & Data Pipelines:** Defining a rigid dataset contract:

$$\text{Raw File} \xrightarrow{\text{Resample}} \text{Mono Conversion} \xrightarrow{\text{Crop/Pad}} \text{Mel Bank} \xrightarrow{\text{dB Scale}} \text{Norm} \rightarrow \mathbf{X} \in \mathbb{R}^{1\times M\times T}$$

1. **Data Integrity & Leakage Prevention:** Ensuring strict group-based splitting (e.g., patient/speaker/device level) to prevent deep learning models from memorizing background noise or recording devices instead of target audio features.

2. Solutions & Exemplar Answers: In-Class Activity

Diagnostic Question (Slide 3)

- **Question:** Two recordings have nearly the same long-term magnitude spectrum. Recording A rises in pitch; Recording B falls. Which representation separates them, and why?

- **Good Answer:** The Short-Time Fourier Transform (STFT).

- *Reasoning:* A global magnitude spectrum (like a standard DFT or Welch estimate) averages energy across time, discarding phase and temporal ordering. The STFT computes windowed spectra over successive time frames m, preserving time localization. In an STFT spectrogram, Recording A will display a diagonal ridge with a positive slope in time-frequency space, whereas Recording B will display a negative slope.

In-Class Activity Prompts (Slide 29)

1. **Predict Frame Count T:**

- *Given Parameters:* $L = 48{,}000$, $N = 512$, $R = 160$, `center=False`.
- *Formula:*

$$T = 1 + \left\lfloor \frac{L - N}{R} \right\rfloor$$

- *Calculation:*

$$T = 1 + \left\lfloor \frac{48{,}000 - 512}{160} \right\rfloor = 1 + \left\lfloor \frac{47{,}488}{160} \right\rfloor = 1 + \lfloor 296.8 \rfloor = 297 \text{ frames} \quad [\text{cite: } 1]$$

1. **Why does `features.squeeze()` break PyTorch `Conv2d` layers?**

- *Good Answer:* Calling an unconstrained `x.squeeze()` removes **all** dimensions of size 1. If the batch size happens to be $B = 1$ during single-sample inference, `squeeze()` strips both the batch dimension and the single channel dimension (e.g., converting $[1,1,64,297] \rightarrow [64,297]$). PyTorch `nn.Conv2d` requires a 4D input tensor shape of `[Batch, Channel, Height, Width]`. Stripping these dimensions throws a tensor dimension mismatch error during forward passes.
- *Fix:* Use explicit dimension squeezing (e.g., `x.squeeze(1)`) or maintain shapes explicitly using `keepdim=True` / `unsqueeze()`.

---

3. Solutions & Exemplar Answers: Weekly Quiz

1. **What information does an STFT preserve that Welch averaging discards?**

- *Answer:* Frame-level time localization and the temporal sequence/order of spectral events.

1. **With n_fft fixed, what does a smaller hop_length change?**

- *Answer:* It increases frame density over time (more frames per second) and increases overlap/redundancy. It does **not** change the frequency bin spacing ($\Delta f = f_s/N$).

1. **Why standardize sample rate before computing features?**

- *Answer:* Because FFT bin frequencies ($f_k = kf_s/N$) and frame spacing ($\Delta t = R/f_s$) depend directly on f_s. Standardizing f_s ensures every bin and frame in the input tensor retains identical physical meaning across all audio files.

1. **What tensor shape should one mono Mel patch have before batching?**

- *Answer:* `[1, n_mels, time_frames]` (Channel × Height × Width).

1. **Power decreases by a factor of 100. What is the dB change?**

- *Answer:* -20 dB.

$$\Delta\text{dB} = 10\log_{10}(0.01) = 10(-2) = -20\text{ dB} \quad \text{[cite: 1]}$$

1. **Why can random validation crops make model comparisons unreliable?**
- *Answer:* Random crops evaluate a different segment of the audio file on every validation epoch, introducing sampling variance/noise directly into the evaluation metric.
1. **Name one shortcut created by zero padding.**
- *Answer:* The model can learn to classify samples based on total clip duration or pad-boundary artifacts (e.g., sudden silence transitions) rather than the underlying acoustic signal.
1. **A patient contributes clips to train and test sets. What failed?**
- *Answer:* **Group-level data splitting failed**. Splitting was executed at the clip level instead of the subject/patient group level, causing information leakage.

---

4. Solutions & Exemplar Answers: Homework Problems

Problem 1: Frame Count & Dimensions

- **Task:** Audio is 2.5 s at 22.05 kHz, $n\_fft = 1024$, $hop = 256$, `center=False`. Find total samples L, frequency bins, and complete frames T.

- **Calculation:**

- $L = \text{round}(2.5 \times 22{,}050) = 55{,}125$ samples.

- Frequency Bins = $\frac{n\_fft}{2} + 1 = \frac{1024}{2} + 1 = 513$ bins.

- $T = 1 + \left\lfloor \frac{55{,}125 - 1024}{256} \right\rfloor = 1 + \left\lfloor \frac{54{,}101}{256} \right\rfloor = 1 + \lfloor 211.33 \rfloor = 212$ frames.

Problem 2: Representation Choice

- **Task:** Compare linear spectrogram vs. Mel spectrogram for detecting a narrow 7.2 kHz machine fault.

- **Exemplar Response:**

- **Choice:** Start with a **Linear-Frequency Spectrogram** (or a custom Mel bank with high-frequency resolution).

- **Justification:** Standard Mel filterbanks compress physical frequencies logarithmically to mimic human auditory perception, resulting in wide pooling filters at higher frequencies. A narrow acoustic artifact at 7.2 kHz would be averaged across a broad Mel band, blurring the fault signature. Linear spectrograms maintain uniform bin width ($\Delta f = f_s/N$) across all frequencies, preserving sharpness at high frequencies.

Problem 3: Crop Policy

- **Task:** Design training and evaluation crops for 30 s clips containing a single 1 s target event.
- **Exemplar Strategy:**
- **Training:** Extract positive training crops centered around target event timestamps (with jitter), alongside labeled negative background crops from the remaining regions. This ensures every extracted training patch contains valid ground-truth labels matching its contents.
- **Evaluation:** Process long 30 s evaluation clips using a deterministic sliding-window pipeline with fixed overlap. Aggregate window logits across time using a max-pooling operation (to detect rare events) and report clip-level metrics.

Problem 4: dB Calculation & Clamping

- **Task:** Convert $\frac{P}{P_{\text{ref}}} = \{1, 0.25, 0.01, 10^{-5}\}$ to dB. Explain clamping.
- **Calculations:**
- $10\log_{10}(1.0) = \mathbf{0}$ dB
- $10\log_{10}(0.25) \approx \mathbf{-6.02}$ dB
- $10\log_{10}(0.01) = \mathbf{-20}$ dB
- $10\log_{10}(10^{-5}) = \mathbf{-50}$ dB
- **Clamping Rationale:** Audio signals frequently contain exact zero-power regions (silence, digital zero padding). Since $\log_{10}(0) \to -\infty$, taking the logarithm without clamping causes `NaN` or `-inf` values in gradient calculations. Values must be clamped to a small positive lower bound (e.g., 10^{-10} or `1e-6`) before log evaluation.

Problem 5: Debug Tensor Shapes

- **Task:** A `DataLoader` returns shape `[16, 64, 297]`, but `nn.Conv2d` expects 4D inputs. Diagnose and repair.

- **Diagnosis:** The custom Dataset `__getitem__` removed or omitted the channel axis (returning `[64, 297]` instead of `[1, 64, 297]`), leading to a 3D batch tensor `[B, M, T]`.

- **Repairs:**

1. **Dataset-level Repair (Best Practice):** Ensure `__getitem__` explicitly returns shape `[1, 64, 297]`:

```
# Inside Dataset __getitem__
mel_spec = mel_transform(waveform)  # Shape: [1, 64, T]
return mel_spec, label
```

1. **Model/Batch-level Repair:** Insert explicit channel expansion in the training loop or model forward function:

```
x = x.unsqueeze(1)  # Reshapes [16, 64, 297] -> [16, 1, 64, 297]
```

Problem 6: Data Leakage Protocol & Auditing

- **Task:** Write a split-and-normalization protocol for a dataset with 5 clips per patient to prevent leakage.

- **Protocol:**

1. Group dataset indices by `patient_id`.

2. Perform Group K-Fold or Stratified Group Split ensuring **all clips from a single `patient_id` belong exclusively to Train, Validation, or Test**.

3. Compute dataset normalization statistics (mean and standard deviation) **exclusively across training set patients**. Apply these pre-computed statistics to validation and test tensors.

- **Auditing Verification Checks:**

- **Check 1 (Intersection Set Check):** Verify that $\text{Set}(\text{Patient\_IDs}_{\text{train}}) \cap \text{Set}(\text{Patient\_IDs}_{\text{test}}) = \emptyset$.

- **Check 2 (Adversarial Group Classifier):** Train a linear probe on model feature embeddings to predict `patient_id`. Near-chance classification accuracy confirms the network is learning generalizable audio signatures rather than patient-specific recording shortcuts.

Week 14, Lecture 28

Explain in more detail what's going on here and what good answers and/or examples are. For all mathematical equations, use standard single dollar signs . .. for inline math and double dollar signs

. ..

for block/display math. Do not use (...) or [...]. Output the raw text without escaping the dollar signs.

Chat GPT (High):

Worked for 38s Week 14, Lecture 28 — CNN Architectures, Training, and Regularization Detailed lecture explanations, mathematical derivations, engineering examples, and model student answers

Course: Digital Signal Processing and Machine Learning Level: Senior undergraduate / first-year graduate ECE References: Oppenheim and Schafer, Discrete-Time Signal Processing; Raschka et al., Machine Learning with PyTorch and Scikit-Learn

This explanation follows your uploaded 45-slide Lecture 28 PowerPoint, including its instructor notes, homework, and weekly quiz.

Lecture 27 established how an audio waveform becomes a physically meaningful log-Mel spectrogram. Lecture 28 addresses what happens next: how a convolutional neural network operates on that spectrogram, how its parameters are trained, how we determine whether it generalizes, and how regularization can help.

The lecture has four major sections:

Time

Subject

Central question

0–25 min

CNN architectures

What spectral and temporal patterns can the network detect?

25–45 min

Training and optimization

How do logits become gradients and parameter updates?

45–60 min

Overfitting and dropout

How do we distinguish successful learning from memorization?

60–75 min

SpecAugment

How can we improve generalization without destroying relevant acoustic information?

A unifying principle is that architecture, optimization, and regularization should reflect the physical properties of the signals and the intended classification task. Part I. Audio CNN Architectures Slide 3 — Opening Diagnostic: 99% Training Accuracy, 58% Validation Accuracy

The slide presents a classifier that fits the training recordings almost perfectly but performs poorly on validation data. Validation loss begins increasing after epoch 7.

Three hypotheses are proposed:

```
A: The model memorizes recording-specific details.

B: The validation transformation is stochastic or inconsistent.

C: The data split leaks information or violates the intended source grouping.
```

The question is not simply which hypothesis is correct, but what evidence could distinguish these explanations. Hypothesis A: Overfitting

Suppose the CNN has enough parameters to memorize acoustic properties of individual recordings.

Training accuracy could approach 99%, while performance on independent recordings remains poor.

Common memorized characteristics include microphone response, background noise, recording-room acoustics, and source-specific artifacts.

The characteristic learning-curve pattern is decreasing training loss accompanied by increasing validation loss after some point. Hypothesis B: Inconsistent validation

Suppose the validation Dataset uses a random three-second crop from each ten-second recording.

The model may receive different validation audio at every epoch.

Consequently, changes in validation loss can reflect changes in the evaluation data rather than changes in the classifier.

An additional problem occurs if dropout remains active or batch-normalization statistics continue updating during validation. Hypothesis C: Improper splitting

If patches from the same recording appear in both training and validation, the two partitions are not independent.

This usually makes validation appear artificially better, rather than directly explaining a low 58% accuracy. However, inconsistent source grouping, class distributions, or collection conditions across splits can also cause poor generalization.

Thus, we should inspect the source IDs and partitioning protocol rather than treating 58% accuracy as proof of leakage. Good student answer

```
I would first make validation deterministic and verify that the model is in
evaluation mode. Next, I would inspect recording and source identifiers to
check whether the partitions provide the intended independence. Finally, I
would compare training and validation learning curves and repeat training
with a smaller model or stronger regularization. If validation remains poor
with a fixed and appropriate split while training fit is nearly perfect,
overfitting is a strong explanation.
```

What makes an excellent answer?

A strong student distinguishes observations from causes.

The 99% versus 58% accuracy gap is evidence of poor generalization under the current protocol, but it does not uniquely identify the cause.

The instructor notes specifically recommend repeatable validation, source-ID inspection, learning curves, and controlled changes to capacity or augmentation. Slide 4 — The CNN Input Contract

The lecture defines the input as

$$ \mathbf X\in\mathbb R^

***** Gets stuck here *****

Week 14, Lecture 28

Explain in more detail what's going on here and what good answers and/or examples are.

ClaudeAI Sonnet 5.5 Medium:

Week 14, Lecture 28: CNN Architectures, Training, and Regularization

The big picture

Lecture 27 ended with a clean `[B, 1, M, T]` log-Mel tensor. This lecture covers what happens after it: a CNN that reads the tensor, a training loop that fits it, and regularization that keeps it from memorizing. The theme is the same as before. Every choice should be traceable to a task assumption or a measured result, and the failures that matter most are silent ones. A leaky split, a stochastic validation transform, or a wrongly applied softmax won't crash anything, but they will give you numbers you can't trust.

The lecture has four layers:

1. **Architecture:** a small Conv2d stack whose kernels act as local spectro-temporal detectors, ending in a head that doesn't depend on clip length.
2. **Training loop:** logits, loss, gradients, and validation done correctly.
3. **Overfitting and dropout:** reading learning curves, then choosing an intervention to match.
4. **SpecAugment:** masking frequency bands and time frames so the model can't lean on one fragile cue.

Section walkthrough

Slides 4–7: Conv2d on a spectrogram. Conv2d slides over the last two axes, Mel frequency and time. Early layers detect motifs: horizontal ridges (tones), vertical edges (onsets), diagonals (chirps), and textures. Deeper layers combine them into class evidence. Kernel shape is an inductive bias, and the axes aren't interchangeable:

Kernel	Looks for
3×3	Local edge or blob
3×9 (short in Mel, long in time)	Stable narrowband event
9×3 (broad in Mel, short in time)	Broadband onset
Dilated 3×3	Longer modulation with sparse sampling

The output-size formula is $\mathrm{Lout} = \lfloor(\mathrm{Lin} + 2p - d(k-1) - 1)/s + 1\rfloor$. With $k=3$, $p=1$, $s=1$, $d=1$ the size is preserved, and each 2×2 pool halves it. Doing this per axis catches most classifier-head bugs before training.

Slides 8–14: the architecture. Three blocks (16, 32, 64 channels, each pooled by 2), then global average pooling to `64×1×1`, then Dropout and `Linear(64, C)`. Channels rise while resolution falls. The head is deliberately tiny:

- 64 → 10 costs 64×10 + 10 = **650** parameters.

- Flattening `64×8×16` into 10 classes would cost 8,192×10 + 10 = **81,930**, roughly 125 times more. That is exactly the memorization pressure the lecture wants to avoid.

The head size isn't the only benefit. A flatten-then-Linear head has `in_features` tied to T, so the model only accepts one clip length. `AdaptiveAvgPool2d((1,1))` turns any `64×H×W` map into `64×1×1`, so the classifier always sees 64 values. (Slide 13 shows T=128 giving 8×16 and T=173 giving 8×21 before pooling, with identical outputs after.)

Slide 10 asks the key question for any pooling choice: does the label depend on where the event occurs, or only that it occurs? Max pooling suits sparse events and average pooling suits distributed texture. Global pooling is a hypothesis that the label ignores position.

Slides 16–24: the training loop.

- `CrossEntropyLoss` takes **raw logits `[B, C]`** and **integer class indices `[B]` of dtype long**. It applies log-softmax internally, so adding your own softmax is redundant and numerically worse.
- One step runs in this order: zero gradients, forward, loss, backward, step. Gradients accumulate by default, which is why zeroing matters.
- `model.train()` versus `model.eval()` matters. Dropout is random in train and off in eval, and BatchNorm updates running statistics in train and uses stored ones in eval. Use `model.eval()` and `torch.inference_mode()` for validation. They do different jobs, since eval changes layer behavior and inference mode turns off autograd.
- Log loss weighted by batch size. An unweighted mean of batch means is wrong when the last batch is smaller.
- AdamW is often fast to a good baseline and decouples weight decay from the gradient update. SGD with momentum is slower and more learning-rate-sensitive but is the classic carefully tuned benchmark. Compare them under the same split, metric, augmentation, and budget.
- Weight decay and learning rate interact, so tune them together on a log-spaced grid (for example 10^{-5} to 10^{-2}). Too little lets weights grow and training fit dominates. Too much underfits.
- Cosine annealing smoothly lowers the learning rate. Step it once per epoch if `T_max` is in epochs.

Slides 25–29: overfitting and dropout. Read the curves before prescribing anything:

- **Healthy:** both losses fall with a small, stable gap.
- **Overfit:** train loss falls while validation loss turns upward. Interventions are more data, augmentation, dropout, weight decay, a smaller model, or early stopping.
- **Underfit:** both stay high. Verify optimization first, then consider more capacity or weaker regularization.

Dropout computes $\tilde{h} = m \odot h/(1-p)$, so survivors are scaled up during training and nothing changes at evaluation. It stops the classifier from relying on one fragile coalition of features. Placement matters. It goes before the head, with p around 0.2–0.5, because dropout too early or too strong destroys local evidence and causes underfitting. And it can't fix a leaky split.

Small datasets need fold discipline. UrbanSound8K has 8,732 excerpts, 10 classes, and 10 supplied folds. ESC-50 has 2,000 clips, 50 classes, and 5 folds. Fragments from the same source are kept in the same fold, so respecting the supplied folds is what prevents source leakage.

Slides 30–35: SpecAugment. Frequency masks zero adjacent Mel channels, which builds invariance to missing bands and narrowband interference. Time masks zero adjacent frames, which builds invariance to partial occlusion. Masking belongs only in the training transform, applied after normalization so that a masked value of 0 equals the mean and not an artificial extreme. Validation is deterministic and unmasked.

Mask width has physical meaning: duration = $W\_t \cdot \text{hop} / f_s$. With 20 frames, hop 160, and 16 kHz, that's 0.20 s. The design test is whether 0.20 s removes one cue or erases the entire event. Augmentation should challenge shortcuts while preserving the label.

Worked answers for the in-class items

Opening diagnostic (99% train, 58% validation, validation loss rising after epoch 7). Three hypotheses, and what evidence separates them:

- **A (memorization):** The curves show classic overfitting. Validation loss was falling and then turns upward. A fix like dropout, augmentation, or early stopping improves validation *and* it responds to dropping capacity. A tiny-subset experiment also shows it: the model can drive training loss to zero on a few batches.
- **B (stochastic or inconsistent validation):** Run validation twice on the same weights. If the metric changes, the transform is random or `eval()` was forgotten. Under B the validation metric is noisy from the start rather than showing a smooth rise after a minimum.
- **C (leakage or grouping violation):** Check source-ID intersections between train and validation. Note that leakage usually makes validation look *better* than reality, so a big gap in the pessimistic direction points to A or B more than C. If you also see high validation and a collapse on a truly held-out test fold, C is the suspect.

A strong answer names the evidence for each hypothesis, not just the hypotheses.

Design checkpoint (slide 14):

- **Siren detection:** Presence matters more than timing, so global average or max pooling is reasonable.

- **Bird call sequence:** Order may carry the class, so keep some time structure, for example pool the frequency axis but preserve a coarse time axis, or use a recurrent or attention layer on top.
- **Machine fault:** A narrow band is diagnostic, so avoid pooling away frequency too early. Use adaptive pooling like `(Fout, 1)` to keep coarse frequency structure.

A strong answer states what the label ignores, then picks the pooling to match.

Activity step 1 (shape for `[8,1,64,173]`):

Stage	Shape
Input	[8, 1, 64, 173]
Block 1 conv / pool	[8, 16, 64, 173] / [8, 16, 32, 86]
Block 2 conv / pool	[8, 32, 32, 86] / [8, 32, 16, 43]
Block 3 conv / pool	[8, 64, 16, 43] / [8, 64, 8, 21]
Adaptive pool	[8, 64, 1, 1]
Flatten, dropout, Linear	[8, 64] → [8, C]

Pooling floors odd sizes (173 → 86 → 43 → 21).

Activity step 3 (remove adaptive pooling): The conv layers still work for any T. The failure is at the `Linear`. For T=128 the flattened width is 64×8×16 = 8,192. For T=173 it is 64×8×21 = 10,752. A Linear built for one width raises a matrix-multiplication shape mismatch for the other. The point is that the first error occurs at the head, not in the conv stack.

Extension (overfit one batch): Turn off dropout and masking, train on a single batch, and confirm loss goes near zero. If it doesn't, something is wrong with the data, targets, or optimizer. Then restore the regularizers.

Slide 24: failure-mode table

Symptom	Likely bug	Check
Loss doesn't fall	Targets or LR wrong	dtype and range of y, tiny-batch overfit
Validation varies wildly	Stochastic eval	`eval()` and transforms
Accuracy high, loss high	Overconfident errors	Inspect logits, calibration
Scheduler ineffective	Wrong stepping unit	Log LR every update

Homework: what a strong answer looks like

1. **Shape audit:** 64×173 → 32×86 → 16×43 → 8×21 (convs preserve size, pools floor). Channels: 16×32×86, 32×16×43, 64×8×21.
2. **Parameter count:** The conv has 32·16·3·3 = 4,608 weights and no bias. BatchNorm has scale and shift for 32 channels = 64 trainable parameters, for **4,672** total. A

great answer notes that BatchNorm's running mean and variance are buffers, not trainable parameters.

3. **Variable width:** Flattened width is $64\cdot 8\cdot\lfloor T/8\rfloor$, so the Linear's `in_features` depends on T. Adaptive pooling maps every `64×H×W` to `64×1×1`, so the Linear always receives 64.
4. **Loss debug:** Two errors: a redundant softmax before `CrossEntropyLoss`, and float one-hot targets instead of integer class IDs (this triggers a different target mode). The fix is raw logits `[B,10]`, labels `[B]` of dtype long, then `CrossEntropyLoss(logits, y)`.
5. **Diagnose curves:** Fix the split and seed, then run three jobs: baseline, dropout only, and SpecAugment only. Compare best validation loss, the train-validation gap, and fold accuracy. Select the epoch by the validation criterion, never by final training loss. Changing one thing at a time is what makes it controlled.
6. **SpecAugment and folds:** $24\cdot 160/16000$ = **0.24 s**. Training uses deterministic log-Mel and normalization plus random crop and masks. Validation uses a deterministic crop or pad with no masks, and exactly one supplied ESC-50 fold is held out for validation.

Quiz: model answers

1. Mel frequency and time.
2. `[B, 64, 1, 1]`.
3. No. Pass raw logits.
4. Zero gradients, forward, loss, backward, optimizer step.
5. Dropout is disabled and all features stay active.
6. Overfitting. One intervention: dropout, SpecAugment, weight decay, a smaller model, more data, or early stopping.
7. $10\cdot 160/16000$ = 0.10 s.
8. They reduce source-related leakage and make results comparable under the dataset's protocol.

Things worth knowing beyond the slides

- **Overfitting vs. leakage:** Both can show a train-validation gap, but they need different fixes. Regularization addresses overfitting and does nothing for a contaminated split. The slide states it plainly: neither dropout nor SpecAugment compensates for an invalid validation protocol.
- **Accuracy vs. loss:** High accuracy with high loss means confidently wrong errors. Accuracy can look fine while the model is badly calibrated, so log both, plus class-sensitive metrics.
- **Why masks go after normalization:** Zero after standardization equals the mean level, which is a neutral fill. Masking before normalization would shift the statistics.

- **BatchNorm and eval:** Forgetting `model.eval()` doesn't just leave dropout on. It also leaves BatchNorm using batch statistics, which makes validation depend on batch composition.

If it would help, I can write the full `AudioCNN` plus training loop with these checks built in, or make a set of practice problems.

Yes, please write the full `AudioCNN` plus training loop with these checks built in

ClaudeAI Sonnet5.5 Medium:

Still installing PyTorch in the sandbox so I can test the code before handing it over; checking on the install now.

The install is still running. While it finishes, I'll write the code.

The PyPI install stalled and doesn't have the disk space for the CUDA build. I'm switching to a CPU-only PyTorch from conda-forge.

The CPU-only PyTorch environment looks like it has finished installing. I'm checking that it imports, then I'll run the script's checks.

All checks pass. Next, the full training run.

I wrote `audio_cnn.py`. I ran it end to end on CPU PyTorch: all eight checks pass and training runs.

It uses synthetic log-Mel data (tone, onset, chirp and texture classes, each with a per-source coloration) so it runs with no audio files. To use real audio, swap `make_synthetic()` for your Lecture 27 Dataset and leave the rest alone.

```
python audio_cnn.py                  # checks, then train / val / test
python audio_cnn.py --checks-only
python audio_cnn.py --no-specaug --dropout 0.0 --weight-decay 0   # baseline
run for HW 5
```

What's in it

- **Model:** `Block` is Conv3×3 (no bias) → BatchNorm → ReLU → MaxPool. `AudioCNN` stacks three blocks (16/32/64 channels), then adaptive pooling, Dropout and `Linear(64, C)`. It returns raw logits, with no softmax. `forward` rejects anything that isn't `[B, 1, M, T]`, so a stray `squeeze()` fails at the boundary with a message saying so.
- **Training loop:** `run_epoch` implements the slide 17–19 rules:
    - **Mode and gradients:** one function trains or evaluates, switching on whether an optimizer is passed. Training zeroes gradients with `set_to_none` before each step. Evaluation runs under `inference_mode` with `eval()`.

    - **Loss and metrics:** loss is weighted by batch size. It also tracks per-class recall and balanced accuracy.
    - **Target checks:** it checks that targets are long `[B]` and in range.
- **`fit`:** AdamW plus cosine annealing stepped once per epoch. The checkpoint is the best validation loss with early stopping, and the test set is touched once, at the end. `diagnose()` prints a learning-curve verdict: healthy, overfit, or underfit against ln(C).
- **SpecAugment:** it runs after normalization and only in the training transform, with `mask_duration_seconds` for the physical width.
- **Group-aware split:** whole sources go to train, val or test before any statistics are computed. Normalization is fit on training data only.

The checks, and how they map to the lecture

Check	Lecture item
T=128 → 8×16 and T=173 → 8×21, both → `[B,64,1,1]`; 3-D input rejected	Slides 13, 36
Conv16→32+BN = 4,672 and head 64→10 = 650 parameters	Slide 9, HW 2
`eval()` is deterministic; `train()` with dropout is not	Slide 19
Train/val/test groups are disjoint	Slides 23, 29
Validation items are identical across calls; SpecAugment varies them	Slide 33
20 / 24 / 10 frames = 0.20 / 0.24 / 0.10 s	Slide 34, HW 6, quiz 7
Cosine LR is non-increasing and decays	Slide 22
One-batch overfit (loss 1.57 → 0.002 with dropout and masking off)	Extension on slide 36

`audit_shapes()` uses forward hooks to print the real shape leaving each stage, which is activity step 2.

Run results, and caveats

- **Default settings:** the curves came out healthy, with train loss 0.22 and val loss 0.15, and test accuracy of 1.00 on all four classes.
- **Baseline for HW 5:** with no SpecAugment, no dropout and no weight decay, `diagnose()` flagged overfitting. Validation loss bottomed at 0.063 and then rose to 0.070.
- **Synthetic data is easy:** the perfect test accuracy comes from 48 test patches of a simple synthetic task. Don't read it as a benchmark, since the point is the pipeline.

- **Noisy early validation:** the first few epochs are very noisy (val accuracy swings between 0.25 and 0.79). With only 18 training groups and BatchNorm still warming up, that is expected. It is also why the checkpoint is chosen by validation loss.
- **Crude overfit heuristic:** the 10% rise rule triggers on tiny differences, as it did in the baseline run above. Treat the verdict as a prompt to look at the curves.

For the HW 5 three-run experiment, run the default, then `--no-specaug` (dropout only), then `--dropout 0.0` (SpecAugment only). Compare the best validation loss and the train-validation gap across them.

Week 14, Lecture 28

Explain in more detail what's going on here and what good answers and/or examples are.

Gemini Auto:

Here is the complete pedagogical breakdown for **Week 14, Lecture 28: Audio CNN Architectures, Training, and Regularization**.

---

1. Deep Dive: What Is Happening in This Lecture?

This lecture transitions from feature extraction (Lecture 27) to **model architecture design, PyTorch training engineering, overfitting diagnostics, and spectrogram augmentation (SpecAugment)** for audio classification.

Key Core Concepts & Mechanics

1. **Spectro-Temporal Feature Extraction via `nn.Conv2d`:** Convolutional filters slide across the 2D Mel-frequency (M) and Time (T) axes. Kernels learn localized spectro-temporal motifs—such as horizontal ridges (steady harmonics), vertical stripes (transients/onsets), and diagonal sweeps (chirps).

2. **Dimension Decoupling via Adaptive Pooling:** Traditional flattening (`nn.Flatten()`) ties input time duration T directly to the classifier head's linear weights (`Linear(64 * H * W, C)`), causing runtime shape errors for variable-length inputs. Using `nn.AdaptiveAvgPool2d((1, 1))` collapses spatial dimensions to a fixed $[B, C, 1, 1]$ tensor regardless of input time width T, enabling variable-length audio processing and drastically reducing parameter counts.

3. **Numerically Stable Loss Interfaces:** Passing raw, unnormalized logits directly into `nn.CrossEntropyLoss(logits, targets)` combines `LogSoftmax` and `NLLLoss` into a single numerically stable operation, preventing overflow/underflow issues caused by manual `Softmax()` calls.

4. **Regularization & SpecAugment:** Applying time and frequency masking (zeroing rectangular blocks along frequency or time axes) directly on log-Mel features during training. This forces the network to learn redundant cues rather than relying on narrow spectral shortcuts or single temporal bursts. Validation data must always remain unmasked and deterministic.

2. Solutions & Exemplar Answers: Opening Diagnostic & In-Class Activity

Opening Diagnostic (Slide 3)

- **Scenario:** Training accuracy reaches 99%, but validation accuracy stalls at 58% and validation loss rises after epoch 7.
- **Question:** Which evidence would distinguish between Hypothesis A (Memorization/Overfitting), Hypothesis B (Stochastic Validation Transform), and Hypothesis C (Source/Group Split Leakage)?
- **Exemplar Answer:**
- **To isolate Hypothesis B (Stochastic Validation):** Inspect the validation transform pipeline. If evaluation mode (`model.eval()`) and deterministic transforms are active, running validation twice on the exact same checkpoint will yield **zero variance** in loss. If validation metrics wobble between passes, data randomness is leaking into validation.
- **To isolate Hypothesis C (Data Leakage):** Run a leak check on recording metadata (e.g., `speaker_id`, `device_id`, or `fold`). If metadata overlaps between splits, the model memorizes environment shortcuts. If train and test splits are strictly disjoint at the source level, then a rising validation loss curve while train loss approaches zero confirms classic **Hypothesis A (Model Overfitting/Memorization)**.

In-Class Activity: Tensor Shape Audit & Debug (Slide 36)

1. **Shape Trace for Input** $[B, 1, 64, 173]$ **through `AudioCNN`:**

- **Input:** $[B, 1, 64, 173]$
- **Block 1 (Conv 1$\to$16, Pool** 2×2**):** $[B, 16, 32, 86]$
- **Block 2 (Conv 16$\to$32, Pool** 2×2**):** $[B, 32, 16, 43]$
- **Block 3 (Conv 32$\to$64, Pool** 2×2**):** $[B, 64, 8, 21]$
- **Adaptive Pool `(1, 1)`:** $[B, 64, 1, 1]$
- **Flatten + Linear Head:** $[B, 64] \rightarrow [B, C]$

1. **Why Removing Adaptive Pooling Breaks Variable Widths ($T = 128$ vs $T = 173$):**

- **At $T = 173$:** Spatial dimensions after Block 3 pool to 8×21, yielding a flattened feature vector of size $64 \times 8 \times 21 = 10{,}752$.
- **At $T = 128$:** Spatial dimensions after Block 3 pool to 8×16, yielding a flattened feature vector of size $64 \times 8 \times 16 = 8{,}192$.
- **Failure Mode:** Without `AdaptiveAvgPool2d((1, 1))`, a fixed linear layer (`nn.Linear(10752, C)`) will throw a runtime shape mismatch error whenever $T \neq 173$.

---

3. Solutions & Exemplar Answers: Weekly Quiz

1. **What two spectrogram axes does `Conv2d` traverse?**

- **Answer:** Mel-frequency (M) and Time (T).

1. **What output shape does `AdaptiveAvgPool2d((1,1))` produce from $[B, 64, H, W]$?**

- **Answer:** $[B, 64, 1, 1]$.

1. **Should softmax be applied before `CrossEntropyLoss` in the standard single-label case?**

- **Answer:** No. PyTorch's `nn.CrossEntropyLoss` expects unnormalized raw logits.

1. **Name the five operations in one training step, in order.**

- **Answer:**

1. Zero gradients (`optimizer.zero_grad(set_to_none=True)`)
2. Forward pass (`logits = model(x)`)
3. Compute loss (`loss = loss_fn(logits, y)`)
4. Backward pass (`loss.backward()`)
5. Optimizer step (`optimizer.step()`)
6. **What changes when `model.eval()` is called for dropout?**

- **Answer:** Dropout is disabled; all feature activations pass through unmasked without random zeroing or scaling.

1. **Train loss falls while validation loss rises. Name the pattern and one intervention.**

- **Answer: Overfitting**. *Interventions:* Add dropout, apply SpecAugment, increase weight decay, reduce model capacity, or apply early stopping.

1. **At 16 kHz and $hop = 160$, how long is a 10-frame time mask?**

- **Answer:** 0.10 seconds (100 ms).

$$\text{Duration} = \frac{W_t \times \text{hop}}{\text{fs}} = \frac{10 \times 160}{16{,}000} = 0.10\text{ s} \quad [\text{cite: 2}]$$

1. **Why should supplied UrbanSound8K or ESC-50 folds be respected?**

- **Answer:** They ensure that audio snippets from the same original source file/recording session remain in the same fold, preventing source-level data leakage and allowing standardized benchmark comparisons.

---

4. Solutions & Exemplar Answers: Homework Problems

Problem 1: Spatial Shape Audit

- **Task:** Input shape $[B, 1{,}64{,}173]$, three 3×3 convs ($p = 1, s = 1$), each followed by 2×2 max pooling. Give spatial dimensions and channel shapes after each block.

- **Exemplar Answer:**

- Convolutions with $k = 3, p = 1, s = 1$ preserve spatial dimensions: $\lfloor (L + 2(1) - 1(3-1) - 1)/1 + 1 \rfloor = L$.

- Max pooling 2×2 uses floor division: $\lfloor L/2 \rfloor$.

- **Block 1 Output:** Spatial shape $= 32 \times 86$, Channel shape $= [B, 16{,}32{,}86]$.

- **Block 2 Output:** Spatial shape $= 16 \times 43$, Channel shape $= [B, 32{,}16{,}43]$.

- **Block 3 Output:** Spatial shape $= 8 \times 21$, Channel shape $= [B, 64{,}8{,}21]$.

Problem 2: Parameter Count

- **Task:** Count trainable parameters in `Conv2d(16, 32, kernel_size=3, padding=1, bias=False)` followed by `BatchNorm2d(32)`.

- **Calculation:**

- **Conv2d parameters:** $C_{\text{out}} \times C_{\text{in}} \times k_h \times k_w + \text{bias} = 32 \times 16 \times 3 \times 3 + 0 = 4{,}608$ parameters.

- **BatchNorm2d parameters:** Learnable affine parameters (γ scale and β shift) per output channel $= 2 \times 32 = 64$ parameters.

- **Total Trainable Parameters:** $4{,}608 + 64 = \mathbf{4{,}672}$ parameters.

Problem 3: Variable Width & Adaptive Pooling

- **Task:** Explain why `Flatten` → `Linear` breaks for variable T and show how `AdaptiveAvgPool2d((1, 1))` fixes it.

- **Exemplar Response:**

- Direct flattening converts spatial maps into $C \times H \times W$ features. Because $W = \lfloor T/\text{stride} \rfloor$, any variation in time frames T changes the total flattened length, causing dimension mismatch inside subsequent fixed-size `nn.Linear(in_features, out_features)` layers.

- `nn.AdaptiveAvgPool2d((1, 1))` dynamically adjusts pooling kernel and stride parameters to aggregate spatial features across any height H and width W into a fixed 1×1 spatial output. The resulting shape is always $[B, C, 1,1]$, providing a static feature vector of length C to `nn.Linear(C, n_classes)` regardless of T.

Problem 4: Loss Debugging

- **Task:** A model applies `Softmax`, produces $[B, 10]$, and receives float target vectors $[B, 10]$. Identify errors and rewrite the standard single-label pipeline.

- **Identified Errors:**

1. Applying `Softmax` before `nn.CrossEntropyLoss` causes double-softmax behavior, compressing gradient scales and stalling learning.

2. Passing float one-hot targets $[B, 10]$ into standard integer-class `CrossEntropyLoss` triggers type/shape mismatches.

- **Corrected Pipeline Code:**

```
# 1. Model output: Raw unnormalized logits of shape [B, C]
logits = model(x)  # No Softmax layer in forward()

# 2. Targets: 1D Tensor of class indices, dtype torch.long, shape [B]
targets = torch.tensor([0, 4, 2, ...], dtype=torch.long)

# 3. Loss function call:
criterion = nn.CrossEntropyLoss()
loss = criterion(logits, targets)
```

Problem 5: Overfitting Curve Diagnostics

- **Task:** Train loss falls $1.8 \rightarrow 0.05$ while validation loss bottoms out at epoch 9 and climbs to 1.4. Propose a controlled 3-run experiment.

- **Experimental Design:**
- **Control Setup:** Hold random seeds, dataset splits, learning rate schedule, and training epochs constant across all runs.
- **Run 1 (Baseline):** Standard model without dropout or SpecAugment.
- **Run 2 (+ Dropout):** Add `nn.Dropout(p=0.3)` prior to the classifier linear head.
- **Run 3 (+ SpecAugment):** Apply random frequency masking ($F = 8$) and time masking ($T = 16$) to the training pipeline.
- **Evaluation Metric:** Measure minimum validation loss, epoch at minimum loss, and generalization gap ($\text{Loss}_{\text{val}} - \text{Loss}_{\text{train}}$). Select the checkpoint based on minimum validation loss rather than final training epoch.

Problem 6: SpecAugment & Fold Protocol

- **Task:** At 16 kHz with $hop = 160$, calculate duration of a 24-frame mask. Design train/validation transforms for one ESC-50 fold.
- **Mask Duration Calculation:**

$$\text{Duration} = \frac{24 \times 160}{16{,}000} = \mathbf{0.24}\ \text{seconds} \quad (240\ \text{ms}) \quad [\text{cite: 2}]$$

- **Transform Protocol:**
- **Training Set (Folds 1–4):**
- Random waveform cropping / padding to 3.0 s.
- Fixed log-Mel spectrogram conversion.
- Normalization using Training Set statistics.
- **Apply SpecAugment:** Random frequency masking ($F_{\text{max}} = 8$) + Random time masking ($T_{\text{max}} = 24$).
- **Validation Set (Fold 5):**
- Center cropping / padding to 3.0 s (deterministic).
- Fixed log-Mel conversion.
- Normalization using Training Set statistics.
- **No SpecAugment / Masking** (deterministic evaluation).

Week 14, Lecture 27 Week 14, Lecture 28

What are good exit ticket questions for these lectures?

ChatGPT (High):

Week 14 — Exit Ticket Questions with Answers

Digital Signal Processing and Machine Learning\ Senior undergraduate / first-year graduate ECE

For these two lectures, I recommend exit tickets that test conceptual understanding, engineering judgment, and the ability to identify mistakes, rather than simply recalling definitions.

I've selected three recommended questions for each lecture, suitable for a 5–7 minute exit ticket, followed by three alternatives that can be rotated between semesters or used as harder conceptual checks.

Lecture 27 — Audio as Images

The main learning objectives are understanding the STFT and Mel representation, choosing preprocessing appropriately, and recognizing errors in audio-data pipelines.

Recommended Exit Ticket

Question 1 — Frequency Resolution Versus DFT Sampling

A student computes the STFT of a signal using a 400-sample Hann window and a 512-point FFT. The student increases `n_fft` to 2048 while keeping the window length unchanged.

Does this improve the ability to distinguish two closely spaced sinusoidal frequencies? Explain.

Good answer:

No. Increasing `n_fft` reduces the spacing between frequency samples of the windowed DTFT, but does not narrow the spectral main lobe associated with the 400-sample window.

For a periodic Hann window, the approximate main-lobe null-to-null width is

$$\Delta f \approx \frac{4F_s}{L}$$

Since the window length is unchanged, its inherent frequency resolution remains unchanged.

Increasing the window length, rather than merely zero-padding, would improve the ability to separate nearby stationary tones.

What to look for: Students should distinguish DFT-bin spacing from physical frequency resolution. A statement such as "2048 points gives four times better resolution" indicates a misconception.

Question 2 — Is a Mel Spectrogram Always Better?

A machine-fault classifier must distinguish a narrow vibration component at 7.2 kHz from one at 7.3 kHz.

Would you automatically choose a Mel spectrogram over a linear-frequency spectrogram? Explain your reasoning.

Good answer:

No. A conventional Mel filter bank combines neighboring frequency bins, with relatively broad frequency bands at higher frequencies.

The two narrowband components may therefore produce very similar Mel features.

A linear-frequency spectrogram would be a reasonable starting point because it retains finer high-frequency spectral detail, subject to choosing an appropriate analysis-window duration.

A sufficiently dense Mel filter bank could also be tested.

What to look for: Students should understand that Mel filtering loses some spectral information and introduces a task-dependent representation bias.

An excellent answer identifies the importance of window length as well as Mel-filter bandwidth.

Question 3 — Data Leakage from Spectrogram Patches

A student has 100 audio recordings. Each recording is divided into 20 overlapping three-second spectrogram patches. The student randomly assigns 80% of the patches to training and 20% to validation.

The CNN achieves 98% validation accuracy.

What is potentially wrong with this experiment, and how would you correct it?

Good answer:

Patches from the same original recording may appear in both training and validation.

They can share waveform samples, background sounds, microphone characteristics, and other recording-specific information.

The CNN may recognize these characteristics instead of learning generalizable acoustic patterns.

The correct procedure is to split independent source recordings, or a higher-level group such as machine or speaker, before extracting patches.

All patches from one group should remain within the same partition.

What to look for: Students must explain why the split is problematic, not merely say "data leakage."

A very good answer identifies that the appropriate grouping level depends on whether the model is intended to generalize to new recordings, new recording sessions, or entirely new machines.

Alternative Questions for Lecture 27

Question 4 — Normalization Can Remove Class Information

Two recordings have identical spectral shapes, but recording B has twice the waveform amplitude of recording A.

Both are converted to fixed-reference log-Mel spectrograms, then independently standardized to zero mean and unit variance.

What useful classification information might be lost?

Good answer:

Doubling waveform amplitude increases spectral power by a factor of four, corresponding to

$$10\log_{10}(4) \approx 6.02.$$

Independent per-patch mean subtraction removes this constant level difference, assuming no flooring or clipping effects.

If loudness distinguishes the classes, the normalization may eliminate essential information.

If microphone gain is a nuisance variable, the same normalization could instead be beneficial.

Concept tested: Normalization determines which signal variations the model treats as irrelevant.

Question 5 — Tensor Shape Debugging

A PyTorch `DataLoader` produces an audio-feature batch with shape `[16,64,297]`.

The first CNN layer is

```
nn.Conv2d(1, 16, kernel_size=3, padding=1)
```

What is wrong with the input tensor? What should its dimensions be?

Good answer:

The input is missing the channel dimension.

The required shape is

```
[16, 1, 64, 297]
```

corresponding to batch, channel, Mel-frequency bins, and time frames.

The Dataset should return individual examples with shape `[1,64,297]`.

The missing dimension can also be restored to the existing batch using

```
x = x.unsqueeze(1)
```

Concept tested: Students understand the representation contract rather than simply memorizing PyTorch syntax.

Question 6 — Cropping Rare Events

A 30-second recording contains one 1-second alarm. A Dataset selects random three-second training crops, assigning every crop the label "alarm."

Why is this potentially incorrect? Suggest a better approach.

Good answer:

Most randomly selected crops may not contain the alarm.

Assigning them all the positive label creates incorrect training examples.

If event timestamps are available, positive training crops should include the event, while verified background crops should be labeled negative.

Validation could use fixed sliding windows and a documented aggregation rule to produce recording-level predictions.

Concept tested: Distinction between recording-level labels and patch-level labels.

Lecture 28 — CNN Architectures, Training, and Regularization

This lecture emphasizes architecture and tensor dimensions, optimization correctness, overfitting, and the physical interpretation of regularization.

Recommended Exit Ticket

Question 1 — What Does Global Average Pooling Really Do?

A CNN processes a log-Mel spectrogram of shape

$$[B, 1, 64, 173].$$

Three convolutional blocks each preserve spatial dimensions during convolution and then apply 2 × 2 max pooling with stride 2. Their output channel counts are 16, 32, and 64.

(a) What is the shape after the third max-pooling layer?

(b) What is the shape after `AdaptiveAvgPool2d((1,1))`?

(c) What information might global average pooling discard?

Good answer:

After three pooling operations, the spatial dimensions are

$$64 \times 173 \ \rightarrow 32 \times 86 \ \rightarrow 16 \times 43 \ \rightarrow 8 \times 21.$$

The output is

⊡

Adaptive average pooling produces

⊡

Global pooling removes explicit information about the spatial locations of activations within each feature map.

This may be desirable when detecting whether a sound occurs anywhere in the recording, but it can be harmful when event order or exact frequency location is important.

What to look for: Full credit requires both correct tensor dimensions and an explanation of the information tradeoff.

Question 2 — Diagnosing Overfitting

A CNN achieves 99% training accuracy but only 65% validation accuracy. Training loss continues decreasing, while validation loss begins increasing after epoch 8.

What is the likely problem? Give one intervention and describe how you would test whether it helps.

Good answer:

The pattern suggests overfitting: the model is fitting the training examples but not generalizing well to independent validation data.

One intervention is dropout, weight decay, SpecAugment, reduced model capacity, or early stopping.

To test the intervention, hold the dataset split, preprocessing, optimizer settings, and evaluation procedure fixed. Change only the selected regularization method, then compare the best validation loss and accuracy.

The student should also verify that validation is deterministic and that related recordings are not improperly split.

What to look for: A good answer provides a diagnosis, a justified intervention, and a controlled method of evaluation.

A weak answer simply lists several regularization methods without explaining how to determine whether they help.

Question 3 — Can SpecAugment Destroy the Label?

A classifier detects an alarm that consists of a single 100 ms acoustic event.

SpecAugment randomly masks up to 30 consecutive time frames. The STFT uses a 160-sample hop and a 16 kHz sampling rate.

Could this augmentation be harmful? Explain.

Good answer:

The maximum nominal mask duration is

$$T = 30\frac{160}{16000} = 0.30.$$

The mask can remove 300 ms of spectrogram information, which is longer than the 100 ms alarm.

If the mask covers the alarm, the resulting spectrogram may contain no evidence of the labeled event.

The augmentation can therefore create misleading training examples.

A better policy would reduce the maximum mask width, constrain masks to preserve the event, or use an augmentation that does not remove class-defining information.

What to look for: Students must connect mask width to physical duration and recognize that augmentation must preserve label validity.

Alternative Questions for Lecture 28

Question 4 — PyTorch Cross-Entropy Debugging

A student writes:

```
probabilities = torch.softmax(model(x), dim=1)
loss = nn.CrossEntropyLoss()(probabilities, y)
```

What should be changed, and why?

Good answer:

The student should pass the raw model logits directly to `CrossEntropyLoss`:

```
logits = model(x)loss = nn.CrossEntropyLoss()(logits, y)
```

PyTorch's cross-entropy loss internally computes the necessary log-softmax operation in a numerically stable way.

For ordinary single-label classification, y should contain integer class indices with shape `[B]` and dtype `torch.long`.

Concept tested: Students understand what the loss function expects and how logits relate to probabilities.

Question 5 — Why Do We Need Both `model.eval()` and `torch.inference_mode()`?

A student argues that calling `model.eval()` automatically disables gradient computation.

Is the student correct? What does each operation do?

Good answer:

No.

`model.eval()` changes the behavior of modules such as dropout and batch normalization.

It disables dropout and makes ordinary BatchNorm layers use stored running statistics.

It does not automatically disable gradient tracking.

`torch.inference_mode()` disables autograd-related work for computations inside its context.

Both are useful for validation.

Concept tested: Separation between model behavior and gradient computation.

Question 6 — Why Can Validation Accuracy Stay Constant While Validation Loss Increases?

During training, validation accuracy remains at 90%, but validation cross-entropy increases steadily.

How can this happen?

Good answer:

Accuracy measures whether the predicted class is correct.

Cross-entropy also depends on the confidence assigned to the true class.

The classifier may become increasingly confident in a few incorrect predictions without changing the total number of correct predictions.

For an example with correct-class probability p_y, cross-entropy is

$$\mathcal{L} = -\log p_y.$$

As p_y approaches zero, the loss becomes very large.

Therefore, increasing confidence in incorrect predictions can increase validation loss while accuracy remains unchanged.

Concept tested: Understanding of cross-entropy beyond its PyTorch implementation.

Recommended Selection and Grading

For a typical 75-minute class, I would use the following format:

Lecture 27 exit ticket

5–7 min

Use Questions 1, 2, and 3 — STFT resolution, Mel representation choice, and data leakage.

Lecture 28 exit ticket

5–7 min

Use Questions 1, 2, and 3 — pooling dimensions and invariance, overfitting diagnosis, and SpecAugment label validity.

I would grade each question out of two points:

Score	Characteristics
2 points	Correct conclusion with a clear mathematical, physical, or statistical explanation
1 point	Correct conclusion but incomplete or weak reasoning
0 points	Incorrect conclusion or fundamental misconception

For multi-part numerical questions, the arithmetic and conceptual explanation should both contribute to the score.

What distinguishes a good exit-ticket answer?

At this level, I would look for three qualities.

First, the answer must identify the governing principle. For example, "zero-padding samples the same DTFT more densely" is substantially better than "the FFT size doesn't matter."

Second, the answer must explain the consequence for the engineering task. For example, "frequency masking may remove a narrowband fault signature" connects augmentation directly to the physical meaning of the label.

Third, the answer should identify an appropriate corrective action or design choice. This might be using a longer analysis window, a group-based dataset split, or a deterministic validation transform.

The two lectures also lend themselves to a useful progression: Lecture 27 asks whether the feature representation is physically meaningful; Lecture 28 asks whether the CNN learns and evaluates that representation correctly.

That connection is what I would most want students to retain at the end of Week 14.

Week 14, Lecture 27

Explain in more detail what's going on here and what good answers and/or examples are.

ChatGPT Medium:

Week 14, Lecture 27: Audio as Images — From DSP to Deep Learning

Detailed lecture explanations, mathematical intuition, examples, and suggested student answers

Course: Digital Signal Processing and Machine Learning\ Audience: Senior undergraduate / first-year graduate ECE students\ Duration: 75 minutes\ Textbook connections: Oppenheim & Schafer (DSP), Raschka (Machine Learning with PyTorch and Scikit-Learn)

This lecture represents an important transition in the course. Up to this point, students have primarily studied signal representations to understand, analyze, or modify signals. Now we want to use those representations as inputs to a machine learning system.

The central question is:

How do we transform a physical audio waveform into a numerical representation suitable for a convolutional neural network without discarding the information needed for classification?

The lecture emphasizes five interconnected ideas:

1. The STFT preserves information about how spectral content changes over time.
2. Spectrograms and Mel spectrograms provide structured, two-dimensional representations that CNNs can process.
3. A PyTorch `Dataset` establishes a consistent contract between physical measurements and network inputs.
4. Cropping, padding, logarithmic compression, and normalization determine what information the classifier can exploit.
5. Proper splitting and reproducible preprocessing are essential for meaningful machine learning results.

The following discussion follows the order of the uploaded Lecture 27 slides, expanding their mathematical basis, practical significance, and classroom questions.

Part I. From global spectra to time-frequency representations (Slides 3–7)

Slide 3: Opening diagnostic — What does a global spectrum lose?

The slide presents two recordings with nearly identical long-term frequency content:

- Recording A begins at a low frequency and rises to a high frequency.
- Recording B begins at a high frequency and falls to a low frequency.

Students are asked which representation distinguishes them.

Consider two discrete-time chirps:

$$x_A[n]=\cos\left(2\pi\left[f_0\frac{n}{f_s}+\frac{\alpha}{2}\left(\frac{n}{f_s}\right)^2\right]\right)$$

$$x_B[n]=\cos\left(2\pi\left[f_1\frac{n}{f_s}-\frac{\alpha}{2}\left(\frac{n}{f_s}\right)^2\right]\right)$$

where $f_1=f_0+\alpha T$, and T is the recording duration.

Their instantaneous frequencies are

$$f_A(t)=f_0+\alpha t$$

and

$$f_B(t)=f_1-\alpha t.$$

One rises while the other falls through the same frequency range.

A global magnitude spectrum can show that both signals contain approximately the same range of frequencies, but it does not directly reveal the order in which those frequencies occur.

This observation can be made even more rigorous by considering time reversal. If

$$x_B[n]=x_A[L-1-n],$$

then the length-L DFTs satisfy

$$X_B[k]=e^{-j2\pi k(L-1)/L}X_A[-k].$$

For real-valued signals, $|X_A[-k]|=|X_A[k]|$, so

$$|X_B[k]|=|X_A[k]|.$$

Thus, the two signals can have exactly the same DFT magnitude despite having opposite temporal order.

Visual interpretation

Recording A: rising chirp

Recording B: falling chirp

Schematic time-frequency representations: the global frequency content may be identical even when the temporal evolution differs.

Good student answer:

An STFT magnitude spectrogram distinguishes the two signals because it retains approximate time localization of the frequency content. The global magnitude spectrum does not tell us when different frequencies occurred. In the spectrogram, the rising chirp produces an upward-sloping ridge, while the falling chirp produces a downward-sloping ridge.

Additional teaching question: Could the complex DFT distinguish the recordings?

Yes. The complex DFT retains phase information, so, in principle, the original finite sequence can be reconstructed by the inverse DFT. It is specifically the global magnitude spectrum, not the complete complex DFT, that loses the temporal ordering information demonstrated here.

This distinction matters because students sometimes incorrectly conclude that the Fourier transform itself discards time information.

Slide 4: Global and local frequency summaries

The slide compares three methods.

Representation	What it computes	What it retains	What it sacrifices
DFT magnitude / periodogram	Frequency content of a record	Global spectral information	Explicit time localization
Welch PSD estimate	Average of windowed periodograms	Statistical description of spectral power	Temporal order of the segments
STFT / spectrogram	Spectrum for each short-time frame	Localized spectral evolution	Fine time resolution and, often, phase

The important difference between the STFT and Welch's method is not necessarily how the individual spectra are calculated.

Both can begin with windowed DFTs of overlapping data segments.

The distinction is what happens after the individual segments are processed.

For the STFT, we retain

$$ X[m,k] $$

for each time frame m.

For Welch's method, we calculate a power estimate for each segment and average over the segments:

$$ \hat S_{xx}[k] = \frac{1}{M}\sum_{m=0}^{M-1}\hat S_{xx}^{(m)}[k]. $$

Here M denotes the number of segments.

A common segment periodogram normalization is

$$ \hat S_{xx}^{(m)}[k] = \frac{1}{f_s\sum_n w^2[n]} \left| \sum_n x[n+mR]w[n]e^{-j2\pi kn/N} \right|^2. $$

The precise scaling depends on whether a one-sided or two-sided PSD is desired.

Averaging reduces the variance of spectral estimates when segments provide sufficiently independent information. However, we no longer know which segment contained a particular event.

Example: Detecting a defective bearing

Suppose a motor produces a constant vibration component at 200 Hz, but a developing fault produces a short burst of energy around 2 kHz once every 500 ms.

A Welch PSD may reveal elevated average power around 2 kHz. An STFT spectrogram can also show the periodic bursts and their timing.

If the classification label depends on the existence of a particular frequency component, a global PSD might be adequate. If it depends on the frequency component's timing or repetition pattern, the STFT is more informative.

Good student answer to "When should timing be discarded?"

Timing can be discarded when the label depends primarily on the long-term spectral distribution and not on the sequence of events. For approximately stationary noise, a PSD can be an effective feature. For speech, chirps, impulsive faults, or other nonstationary phenomena, time-frequency localization is usually more useful.

Slide 5: STFT — Windowed Fourier analysis

The slide introduces the STFT:

$$ X[m,k] = \sum_n x[n]w[n-mR]e^{-j2\pi kn/N}. $$

Here:

- $x[n]$ is the sampled waveform.
- $w[n]$ is an analysis window.
- m identifies the time frame.
- R is the hop length in samples.
- N is the FFT length.
- k identifies a frequency bin.

An equivalent implementation-oriented form is

$$ X[m,k] = \sum_{r=0}^{L_w-1} x[mR+r]w[r]e^{-j2\pi kr/N}, $$

where L_w is the window length and $N\geq L_w$. The two expressions differ by a frame-dependent phase convention, but give the same magnitude when used consistently.

The STFT effectively performs the following operation:

1. Select a short segment of the waveform.
2. Multiply it by a window.
3. Compute its DFT.
4. Shift the window by R samples.
5. Repeat the process and stack the spectra.

For a sample rate f_s,

$$ \Delta f=\frac{f_s}{N} $$

is the spacing between DFT frequency samples, while

$$ \Delta t=\frac{R}{f_s} $$

is the spacing between frames.

Numerical example

Suppose

$$ f_s=16,000 \text{Hz},\quad N=512,\quad R=160. $$

Then

$$ \Delta f=\frac{16,000}{512}=31.25 \text{Hz} $$

and

$$ \Delta t=\frac{160}{16,000}=0.010 \text{s}. $$

So the STFT produces a new frequency column every 10 milliseconds.

If the window also has 512 samples, its duration is

$$ T_w=\frac{512}{16,000}=32 \text{ms}. $$

This is a good point to emphasize a distinction students often miss:

The window duration is 32 ms, but the frame interval is only 10 ms.

Frames overlap substantially. Each spectrogram column describes a 32 ms section of the waveform, and adjacent columns are only 10 ms apart.

The overlap is

$$ 1-\frac{160}{512}=0.6875=68.75\%. $$

Good student answer: Why isn't the frame interval the same as the window duration?

The window length controls how many samples contribute to each spectral estimate. The hop length controls how far the window advances before computing the next estimate. Overlap allows spectral estimates to be updated more frequently than the window duration.

Slide 6: Time-frequency resolution and redundancy

The major conceptual issue here is the tradeoff between time localization and frequency resolution.

A longer observation interval generally permits better distinction between nearby frequencies, while a shorter observation interval permits better localization of rapid changes.

However, three parameters must be distinguished carefully.

1. Window length

The window length L_w determines the duration of data being analyzed.

A longer window generally produces a narrower spectral main lobe.

For a rectangular window, the approximate separation of its first spectral nulls is

$$ \Delta f_{\text{null-to-null}}\approx\frac{2f_s}{L_w}. $$

For a Hann window, it is approximately

$$ \Delta f_{\text{null-to-null}}\approx\frac{4f_s}{L_w}. $$

These are main-lobe widths, not exact universal criteria for resolving two tones.

The window also determines sidelobe behavior, which is important when a weak component lies near a strong component.

2. Hop length

Reducing the hop length increases the number of time frames but does not fundamentally increase the frequency resolution of each frame.

For example, reducing R from 160 to 80 at 16 kHz changes the frame spacing from 10 ms to 5 ms.

It approximately doubles the number of spectrogram columns, but each still contains information from the same window duration.

3. FFT length

Increasing the FFT length N, while keeping the window length fixed, adds zero padding.

For example, if a 512-sample window is transformed with

$$ N=2048, $$

the DFT-bin spacing becomes

$$ \Delta f=\frac{16,000}{2048}=7.8125 \text{Hz}. $$

But the physical observation interval is still 32 ms.

The additional DFT values describe more densely spaced samples of the same windowed DTFT; they do not provide new independent frequency information.

Good student answer:

A longer window generally improves the ability to separate nearby sinusoids, but reduces the ability to localize rapid changes. A shorter hop increases temporal sampling density and redundancy without changing the window's intrinsic frequency resolution. A larger FFT can provide a smoother spectral display through zero padding, but does not overcome limitations imposed by the window duration.

Useful conceptual trap:

A student says, "We can always get better frequency resolution by increasing `n_fft`."

This is false if the number of observed data samples remains fixed. Increasing the FFT length cannot recover information that was not present in the observation interval.

Slide 7: Welch and STFT answer different questions

Students should understand that Welch and STFT are not competing algorithms in the sense that one is universally better.

They support different objectives.

For stationary vibration monitoring, the primary question might be:

> What is the average power distribution over frequency?

For speech recognition, it is more likely:

> How does the frequency distribution evolve as phonemes and syllables occur?

Welch is often preferable when a low-variance estimate of a stationary PSD is desired.

The STFT is preferable when local events, changing tones, or temporal relationships matter.

A useful question for class is:

If we average all the power spectrogram columns, do we recover Welch's method?

A good answer is: essentially yes, provided the same segmentation, windows, scaling, and averaging conventions are used. Welch's estimate is the average of the appropriately normalized segment periodograms.

This also helps connect spectral estimation, which students already know, to neural-network feature extraction.

Part II. Spectrograms as images (Slides 8–13)

Slides 8–9: From linear frequency to Mel frequency

The Mel spectrogram is one of the most important bridges between classical DSP and modern audio classification.

Students should understand that Mel scaling is not simply a prettier way of plotting the spectrum.

It is a deliberate transformation that changes the representation of spectral information.

In a conventional DFT, frequency bins are uniformly spaced:

$$ f_k=\frac{kf_s}{N}. $$

For example, with $f_s=16$ kHz and $N=512$, consecutive bins are separated by 31.25 Hz throughout the spectrum.

The Mel scale is approximately linear at low frequencies and logarithmic at high frequencies.

One commonly used conversion is

$$ m(f)=2595\log_{10}\left(1+\frac{f}{700}\right). $$

The inverse is

$$ f(m)=700\left(10^{m/2595}-1\right). $$

The purpose is to allocate more representational detail to lower frequencies and less detail to higher frequencies, broadly following properties of human pitch perception.

This is a prior assumption about which spectral distinctions matter.

What does the Mel filter bank actually do?

Let

$$ P[m,k]=|X[m,k]|^2 $$

be the STFT power spectrum.

A Mel filter bank contains M frequency-dependent weighting functions $H_\ell[k]$, where ℓ identifies the Mel band.

The power in Mel band ℓ is

$$ E[m,\ell] = \sum_{k=0}^{N/2}H_\ell[k]P[m,k]. $$

Each Mel band collects a weighted combination of neighboring DFT bins.

The filters are typically triangular and overlap one another.

Illustration: overlapping triangular Mel filters

Conceptual filter-bank illustration, not exact numerical Mel-filter coefficients. Higher-frequency filters cover progressively wider frequency intervals.

The critical point is that Mel filtering is a weighted frequency aggregation operation.

It is not selecting one FFT bin per Mel band, and it does not create any new frequency information.

For a power spectrum containing 257 bins, a 64-band Mel transform changes the number of frequency features from 257 to 64 for each time frame.

It is a form of dimensionality reduction.

Why might this help machine learning?

Suppose we are classifying spoken words.

Small variations in high-frequency spectral details might arise from microphone placement, speaking style, or recording conditions without changing the identity of the word.

Mel filtering can reduce sensitivity to such variations.

It can also reduce the number of inputs to a classifier, limiting computational cost.

However, it can discard information that is crucial for some tasks.

Example: Machine-fault detection

Suppose healthy and defective bearings produce narrow spectral lines at 7,200 Hz and 7,280 Hz, respectively.

A conventional linear-frequency spectrogram may represent these components separately.

A coarse Mel filter bank may combine them into nearly the same feature pattern.

If distinguishing those frequencies is important to the classification task, the Mel representation may be inferior.

Good student answer: Why should a Mel spectrogram not always be preferred to a linear spectrogram?

Mel filtering incorporates a perceptual weighting that is useful for speech and many environmental sound tasks. However, it reduces frequency detail, particularly at high frequencies. In applications involving narrow spectral components or precise physical frequency measurements, a linear-frequency spectrogram may retain more useful information.

One implementation detail worth emphasizing: the exact Mel filters depend on the convention, such as HTK or Slaney, the frequency range, and the filter normalization. Two programs that both calculate "64 Mel bands" may not produce identical results.

Slide 10: Waveform to log-Mel representation

The slide shows the sequence

$$ x[n] \longrightarrow X[m,k] \longrightarrow |X[m,k]|^2 \longrightarrow E[m,\ell] \longrightarrow 10\log_{10}(E[m,\ell]). $$

Each stage transforms the representation and changes what information remains.

Operation	What it accomplishes	Information consequence
STFT	Represents local frequency content	Retains complex local spectral information
Squared magnitude	Computes local power-like values	Discards STFT phase
Mel filter bank	Aggregates neighboring frequency bins	Reduces frequency detail
Logarithm / dB	Compresses dynamic range	Changes scale, potentially discards low-level detail through flooring

This is an excellent slide for discussing the distinction between information preservation and task-relevant information.

Students should not assume that losing information is necessarily undesirable.

In machine learning, the best feature representation is not always the one that retains the largest amount of information.

It is the one that retains information useful for the prediction while reducing sensitivity to irrelevant variation.

Why discard phase?

The STFT contains magnitude and phase:

$$ X[m,k]=|X[m,k]|e^{j\phi[m,k]}. $$

A power spectrogram retains

$$ |X[m,k]|^2 $$

but not $\phi[m,k]$.

For many environmental sound classification tasks, the local distribution of spectral energy is informative enough that phase can be discarded.

However, phase matters in applications such as waveform reconstruction, beamforming, sound localization, and some coherent signal processing tasks.

A subtle but important point is that discarding phase does not necessarily remove all temporal information. The arrangement of energy across successive spectrogram columns can still preserve substantial time structure.

Good student answer: What information has been lost by the time the waveform becomes a log-Mel spectrogram?

The representation has discarded explicit complex phase, aggregated frequency bins into perceptually motivated bands, and often suppressed low-energy information through numerical flooring or dynamic-range limiting. It still retains a coarse description of how spectral energy changes over time.

Why use a logarithm?

Audio power can vary over many orders of magnitude.

Suppose three components have powers

$$P_1=1,\qquad P_2=10^{-2},\qquad P_3=10^{-4}.$$

On a linear scale, the second and third values are very small relative to the first.

On a decibel scale, relative to $P_{\mathrm{ref}}=1$,

$$P_{1,\mathrm{dB}}=0,$$

$$P_{2,\mathrm{dB}}=-20,$$

$$P_{3,\mathrm{dB}}=-40.$$

The logarithmic representation makes differences among weak components easier to express numerically.

A CNN can then operate on features whose numerical dynamic range is more manageable.

Slide 11: A spectrogram becomes a one-channel tensor

This slide illustrates the direct relationship between signal processing and the PyTorch convention for image tensors.

An ordinary grayscale image can be represented by a two-dimensional matrix.

A Mel spectrogram can also be represented by a matrix, with

- vertical dimension: Mel frequency bands;
- horizontal dimension: time frames;
- matrix values: spectral energy, often on a logarithmic scale.

For example,

$$S\in\mathbb{R}^{64\times297}.$$

The network needs an explicit channel dimension:

$$S_{\mathrm{input}}\in\mathbb{R}^{1\times64\times297}.$$

The leading 1 denotes one input channel, not one example in a batch.

When eight examples are assembled into a batch,

$$X\in\mathbb{R}^{8\times1\times64\times297}.$$

Interpreting a CNN input tensor

[B, C, M, T] = [8, 1, 64, 297]

Each layer represents one single-channel spectrogram. The batch dimension is separate from the channel dimension.

For a PyTorch `nn.Conv2d`, the input convention is

```
[batch, channels, height, width]
```

In this application,

```
[batch, channels, mel_bins, time_frames]
```

A simple demonstration is

```
import torchfrom torch import nnx = torch.randn(8, 1, 64, 297)conv =
nn.Conv2d(    in_channels=1,    out_channels=16,    kernel_size=3,
padding=1)y = conv(x)print(x.shape)print(y.shape)
```

Expected output:

```
torch.Size([8, 1, 64, 297])
torch.Size([8, 16, 64, 297])
```

The convolution creates 16 feature maps from one input spectrogram.

Each output feature map contains the response of a learned detector applied across the time-frequency plane.

Good student answer: What does a 3×3 kernel examine?

It examines a local neighborhood containing three adjacent Mel bands and three adjacent time frames. It learns weights that respond to particular local patterns of spectral energy. The same learned weights are applied across the entire spectrogram.

Slide 12: The image analogy helps — with limits

This is an especially important conceptual slide.

A CNN was originally motivated by spatial structure in images.

But a spectrogram is not a conventional photograph.

In a photograph, horizontal and vertical translations often preserve an object's identity.

For example, a photograph of a dog remains a photograph of a dog if the dog shifts 20 pixels to the right.

In a spectrogram, however, the two axes have different meanings.

A horizontal translation represents a shift in time.

A vertical translation represents a shift in frequency, or a shift across Mel bands.

These are not generally equivalent transformations.

Time translation

Suppose a recording contains a door slam.

If the door slam occurs 0.5 seconds later, it is still a door slam.

For clip-level classification, a time shift may preserve the label.

Thus, a CNN's approximate translation equivariance along the time axis can be useful.

Frequency translation

Now consider a sound consisting of a 500 Hz tone.

Shifting this component upward to 1,000 Hz changes its physical frequency.

For some classification tasks, the label may remain unchanged. For example, we might still call both recordings a "tone."

But in machinery monitoring, a 500 Hz component and a 1,000 Hz component may indicate different operating conditions.

Therefore, frequency translation is not automatically a label-preserving transformation.

The same issue arises with frequency-axis pooling. Pooling encourages the classifier to be insensitive to small shifts in frequency location, which may or may not be desirable.

A strong student answer would be:

A CNN is appropriate because spectrograms have local structure, and convolution can detect patterns such as harmonics, onsets, and frequency sweeps. However, the axes are not interchangeable. Time shifts often leave an event class unchanged, whereas frequency shifts can alter pitch or the physical meaning of a signal. Kernel sizes, pooling, and augmentation should reflect these differences.

A useful extension for senior and graduate students is to explain that a 3×3 kernel does not correspond to a fixed square region in physical units. Its temporal width depends on the hop length, and its frequency width in hertz depends on the Mel-band location.

Slide 13: Local spectrogram patterns a CNN can learn

The slide identifies four common patterns.

Horizontal ridge

Steady tone or harmonic

Vertical stripe

Impulse or onset

Diagonal ridge

Chirp or frequency sweep

Broad texture

Noise-like signal

A short convolutional kernel can recognize small segments of these structures.

A horizontal ridge can indicate a persistent narrowband tone.

A vertical stripe can indicate a sudden broadband event.

A diagonal ridge indicates changing frequency.

A diffuse, broadband texture can indicate noise.

However, a single 3×3 kernel only examines a small local area. It may recognize a small segment of a ridge rather than an entire chirp.

Deeper convolutional layers combine simpler patterns into more complex structures.

For example:

- First layer: local edges and changes in spectral energy.
- Second layer: combinations of nearby edges and short ridges.
- Later layers: longer patterns such as harmonic sequences or chirps.

Good student answer: Why are multiple CNN layers useful?

The receptive field increases through successive convolutional layers. Early layers detect local patterns, while deeper layers combine them to recognize structures spanning larger time and frequency regions.

This leads naturally into Lecture 28, where the network architecture, training, and regularization are developed.

Part III. The PyTorch audio pipeline (Slides 14–20)

Slides 14–16: Dataset, DataLoader, and the input contract

The key idea is that the model should not have to worry about the physical characteristics of each individual audio file.

A collection of recordings may have different:

- sampling frequencies;
- numbers of channels;
- recording durations;
- amplitude scales;
- file formats.

But a typical CNN expects every example in a batch to have the same tensor shape and a consistent interpretation of its values.

The slide calls this the representation contract.

For example, a Dataset might promise that every call to `__getitem__()` returns a spectrogram with:

$$\text{shape}=[1,64,297].$$

This means that each record has already been converted to mono, resampled to 16 kHz, cropped or padded to three seconds, processed into 64 Mel bands, and represented using 297 time frames.

The distinction between `Dataset` and `DataLoader` is important.

A `Dataset` defines how to obtain and transform one example.

A `DataLoader` controls how examples are requested, optionally shuffled, loaded in parallel, and combined into batches.

A neural network consumes those batches.

Example: A minimal PyTorch Dataset

```
import torch
from torch.utils.data import Dataset


class MelPatchDataset(Dataset):
    def __init__(self, records, mel_transform, to_db):
        self.records = records
        self.mel_transform = mel_transform
        self.to_db = to_db

    def __len__(self):
        return len(self.records)

    def __getitem__(self, idx):
        record = self.records[idx]
        waveform = record["waveform"]
        label = record["label"]
        mel_power = self.mel_transform(waveform)
        features = self.to_db(mel_power)
        return features, label
```

This example illustrates the object-oriented interface, but deliberately assumes the waveform has already been standardized. It is not yet the complete pipeline shown in the lecture.

A production Dataset would additionally handle sample-rate conversion, cropping, padding, and normalization.

Good student answer: Why put preprocessing inside a Dataset instead of directly in the CNN?

The Dataset separates data preparation from model computation. This creates a consistent input contract, supports reproducible preprocessing, and allows the same CNN architecture to be trained on different data collections. It also makes preprocessing bugs easier to isolate.

An important nuance is that preprocessing need not always be implemented literally inside `__getitem__`. Some transformations may be precomputed or performed in batched GPU operations. The essential principle is that the responsibility and interface remain explicit.

Slide 17: Loading audio and converting to mono

The slide shows:

```
waveform, sr = torchaudio.load(path)
```

For a waveform containing C channels and L samples per channel, the usual output shape is

```
[C, L]
```

For stereo audio:

```
[2, L]
```

The slide converts stereo to mono using

```
waveform = waveform.mean(dim=0, keepdim=True)
```

Mathematically,

$$ x_{\text{mono}}[n] = \frac{x_L[n]+x_R[n]}{2}. $$

The `keepdim=True` parameter is important because it preserves the channel dimension.

The output shape becomes

```
[1, L]
```

rather than

[L]

Why could averaging two audio channels be a problem?

Suppose

$$x_L[n]=\cos(\omega_0 n)$$

and

$$x_R[n]=-\cos(\omega_0 n).$$

Then averaging produces

$$x_{\text{mono}}[n]=0.$$

The signal is completely canceled.

While complete antiphase cancellation is an extreme example, partial cancellation can occur in real multichannel recordings.

Moreover, some tasks depend on spatial information.

For instance, identifying the direction of a sound source may require retaining interchannel time differences and phase differences.

Good student answer: When should stereo be preserved?

Stereo should be preserved when spatial information is relevant to the prediction, such as sound localization, multichannel beamforming, or recognition tasks where channel relationships contain useful information. If the task only requires identifying the type of sound, mono conversion may simplify the model, provided the conversion does not introduce unacceptable cancellation or distortion.

Slide 18: Sample-rate standardization fixes physical meaning

This slide is particularly important from a DSP perspective.

The same tensor index does not necessarily represent the same physical frequency if recordings use different sampling frequencies.

Recall

$$f_k=\frac{kf_s}{N}.$$

Suppose we use an FFT length of 512 and examine bin 32.

For $f_s=16$ kHz:

$$f_{32}=\frac{32(16,000)}{512}=1000 \text{Hz}.$$

For $f_s=48$ kHz:

$$ f_{32}=\frac{32(48,000)}{512}=3000 \text{Hz}. $$

The same FFT bin represents entirely different physical frequencies.

Similarly, with a hop of 160 samples:

$$ \Delta t=\frac{160}{16,000}=10 \text{ms} $$

at 16 kHz, but

$$ \Delta t=\frac{160}{48,000}=3.33 \text{ms} $$

at 48 kHz.

If we blindly process recordings at their original sample rates, a CNN may see feature maps with inconsistent physical meanings.

Resampling to a standard sample rate solves this problem.

Example: Resampling from 48 kHz to 16 kHz

```
import torchaudio

ORIG_SAMPLE_RATE = 48_000
TARGET_SAMPLE_RATE = 16_000

resampler = torchaudio.transforms.Resample(
    orig_freq=ORIG_SAMPLE_RATE,
    new_freq=TARGET_SAMPLE_RATE,
)
waveform_16k = resampler(waveform_48k)
```

The sample-rate conversion ratio is

$$ \frac{16,000}{48,000}=\frac13. $$

This is a decimation operation by a factor of three.

To avoid aliasing, the input must be low-pass filtered before reducing the sampling rate.

The new Nyquist frequency is

$$ f_{\mathrm{Nyquist}}=8000 \text{Hz}. $$

Therefore, frequency components above 8 kHz must be sufficiently attenuated before decimation.

The resampling transform handles the necessary bandlimited interpolation/filtering approximately.

Good student answer: Why must sample-rate standardization occur before Mel feature extraction?

The FFT frequency bins, STFT frame spacing, and Mel-filter frequencies depend on the sample rate. Standardizing the waveform sample rate ensures that a particular time-frequency location has the same physical meaning across examples.

One additional qualification: resampling to 16 kHz cannot restore high-frequency information missing from a recording originally sampled at 8 kHz. The choice of target sample rate must respect the frequency content needed for the classification task.

Slide 19: Fixed-duration patches make batching possible

The slide uses a three-second patch at 16 kHz.

The number of samples is

$$L_{\mathrm{target}}=3(16,000)=48,000.$$

Three cases must be handled.

Case 1: The recording is longer than 48,000 samples

Choose a crop of exactly 48,000 samples.

For training, this may be a random crop.

For validation, it may be a center crop or one of several predetermined windows.

Case 2: The recording contains exactly 48,000 samples

Return it unchanged.

Case 3: The recording is shorter than 48,000 samples

Pad it to the desired length.

Here is a simple implementation:

```
import torch
import torch.nn.functional as F

def crop_or_pad(waveform, target_samples, training=True):
    L = waveform.shape[-1]
    valid = min(L, target_samples)
    if L > target_samples:
        if training:
            start = torch.randint(0, L - target_samples + 1, (1,)).item()
        else:
            start = (L - target_samples) // 2
        waveform = waveform[..., start:start + target_samples]
    elif L < target_samples:
        waveform = F.pad(waveform, (0, target_samples - L))
    return waveform, valid
```

```
import torch
import torch.nn.functional as F


def crop_or_pad(
    waveform: torch.Tensor,
    target_samples: int,
    training: bool = True,
) -> tuple[torch.Tensor, int]:
    """Crops or right-pads an audio waveform to a fixed number of samples."""
    num_samples = waveform.shape[-1]
    valid_samples = min(num_samples, target_samples)

    if num_samples > target_samples:
        if training:
            max_offset = num_samples - target_samples + 1
            start = torch.randint(0, max_offset, (1,)).item()
        else:
            start = (num_samples - target_samples) // 2

        waveform = waveform[..., start : start + target_samples]

    elif num_samples < target_samples:
        pad_amount = target_samples - num_samples
        waveform = F.pad(waveform, (0, pad_amount))

    return waveform, valid_samples
```

The output length is always fixed.

However, the most important issue is not the tensor shape. It is the validity of the label.

Suppose a 30-second recording contains a one-second gunshot-like impulse at the 20-second mark.

The recording is labeled "impulsive event."

If we randomly select a three-second crop from the beginning, the resulting patch might contain no impulse at all.

Nevertheless, if we inherit the full recording label, the Dataset assigns a positive class to background audio.

This is label noise.

The lecture's central principle is:

The label must be valid for the patch being returned, not merely for the original recording.

Good student answer: How should positive training crops be generated for short events?

If event timestamps are available, positive crops should be selected so they contain the labeled event. Background-only crops should be labeled appropriately when the task includes a background or negative class. Otherwise, assigning the full recording label to every crop can create incorrect training examples.

Slide 20: Training and evaluation need different crop policies

Random cropping can be a useful training augmentation because it exposes a classifier to different portions of a recording.

For example, a ten-second recording may contain the same sound type throughout. Random three-second crops show different parts of that recording while retaining a valid label.

But random cropping is problematic during validation and testing.

Why?

Suppose a classifier is evaluated on a 30-second recording containing one brief event.

On the first validation run, a randomly selected three-second crop contains the event.

On the second validation run, the crop misses it.

The two runs can produce different accuracy results even though the model parameters are identical.

This makes model comparison unreliable.

Better evaluation strategies

A center crop is deterministic, but it can miss events outside the center.

For recordings much longer than the patch duration, fixed-window or sliding-window evaluation is often better.

For example, a 30-second recording could be divided into ten nonoverlapping three-second windows.

Let $z_{j,c}$ be the predicted logit for class c from window j.

One aggregation strategy for rare-event detection is

$$z_{\mathrm{clip},c} = \max_j z_{j,c}.$$

This gives a high clip-level score if at least one window strongly supports the class.

However, maximum aggregation can be sensitive to false positives. Alternatives include mean aggregation, top-K pooling, and learned aggregation.

Averaging logits is also not mathematically equivalent to averaging probabilities:

$$\operatorname{softmax}\left(\frac1J\sum_j z_j \right) \ne \frac1J\sum_j\operatorname{softmax}(z_j).$$

The aggregation rule therefore needs to be specified and evaluated.

Good student answer: Design a deterministic evaluation policy for a 30-second recording.

Divide the recording into fixed three-second windows, apply identical feature extraction to each window, compute a prediction for every window, and combine the window scores using a predetermined rule. For rare events, a maximum-type aggregation may be reasonable, but it should be validated against false positives and the evaluation objective.

Part IV. Padding, logarithmic compression, and normalization (Slides 21–23)

Slide 21: Padding creates data and possible shortcuts

Padding is easy to implement numerically but has implications for machine learning.

Suppose a recording contains 1.2 seconds of audio and must be converted into a three-second patch.

At 16 kHz:

$$L_{\mathrm{valid}}=1.2(16,000)=19,200.$$

The target is 48,000 samples.

Therefore, the number of padded samples is

$$L_{\mathrm{pad}}=48,000-19,200=28,800.$$

More than half the resulting waveform consists of padding.

A CNN may learn to recognize where the real audio stops rather than what the real audio contains.

For example, if all clips in class A are about one second long and clips in class B are about three seconds long, padding introduces a strong correlation between label and duration.

The model may achieve high accuracy simply by identifying the padding boundary.

This is called a shortcut feature: a statistically predictive feature that does not reflect the phenomenon the classifier is intended to recognize.

There are several possible approaches.

One is to retain the original valid length as metadata.

Another is to build a mask identifying valid portions of the feature representation.

For example:

```
valid_samples = 19200
n_fft = 512
hop_length = 160
valid_frames = 1 + max(0, (valid_samples - n_fft) // hop_length)
```

For this example,

$$T_{\mathrm{valid}} = 1+\left\lfloor\frac{19,200-512}{160}\right\rfloor = 117.$$

The padded waveform has 297 complete STFT frames under `center=False`, but only the first 117 frames use exclusively original samples. Frames near the original waveform boundary may contain both valid audio and padding.

A Boolean mask can identify the fully valid frames:

```
mask = torch.arange(297) < valid_frames
```

The resulting mask has shape

```
[297]
```

But supplying a mask is only useful if the model or loss function actually uses it.

A standard `nn.Conv2d` does not automatically ignore masked spectrogram regions.

Good student answer: Why might zero padding reduce generalization?

The classifier can use the length or boundary of padded regions as an indirect indicator of the class. This can produce deceptively good validation results if recording duration is correlated with labels in the dataset. The relationship may not hold for new recordings, causing poor generalization.

Slide 22: Power-to-dB conversion compresses dynamic range

The decibel representation is defined relative to a reference power:

$$P_{\mathrm{dB}} = 10\log_{10}\left(\frac{P}{P_{\mathrm{ref}}}\right).$$

The logarithm is base 10.

This should be familiar to students from signals, systems, and communications.

Consider a few ratios.

P/P_{ref}	dB
10	+10
1	0
0.25	−6.02

P/P_{ref}	dB
0.1	−10
0.01	−20
0.001	−30
0.00001	−50

A factor of 10 change in power corresponds to 10 dB.

A factor of 100 change in power corresponds to 20 dB.

Why do we use 10 for power and 20 for amplitude?

For a fixed impedance or otherwise consistent proportionality,

$$P\propto A^2.$$

Therefore,

$$10\log_{10}\left(\frac{P_1}{P_0}\right) = 10\log_{10}\left(\frac{A_1^2}{A_0^2}\right) = 20\log_{10}\left(\frac{A_1}{A_0}\right).$$

So 10 is used for power ratios, and 20 for amplitude ratios.

Why clamp the power before taking its logarithm?

If $P=0$,

$$\log_{10}(0)=-\infty.$$

A tensor containing negative infinity can cause numerical problems during normalization or neural-network training.

A common solution is

$$P_{\mathrm{dB}} = 10\log_{10} \left(\frac{\max(P,\epsilon)}{P_{\mathrm{ref}}} \right),$$

where ϵ is a small positive constant.

For example:

```
eps = 1e-10
power_db = 10 * torch.log10(power.clamp_min(eps))
```

If a reference other than 1 is used, the appropriate reference division must also be included.

An important implementation subtlety is that `torchaudio.transforms.AmplitudeToDB` can use a maximum derived from the input tensor. If a reference or top-dB floor is based on each individual patch, the same physical power can map to different dB values in different examples.

That may be desirable if absolute recording gain is a nuisance, but undesirable if absolute level carries information about the label.

Good student answer: Why is dB conversion useful before CNN training?

The logarithm compresses the large dynamic range of spectral power. It prevents strong spectral components from dominating the numerical scale as much and makes weaker components more distinguishable in the feature representation. However, the reference, floor, and dynamic-range conventions must be applied consistently.

Slide 23: Normalization choices encode different invariances

This slide introduces a deeper machine learning concept: normalization is not just a numerical convenience.

Normalization can also determine which signal properties the classifier is sensitive to.

Method 1: Per-patch normalization

For one spectrogram S, compute

$$ \mu_S=\frac{1}{MT}\sum_{\ell,m}S[\ell,m] $$

and

$$ \sigma_S= \sqrt{ \frac{1}{MT}\sum_{\ell,m}(S[\ell,m]-\mu_S)^2 }. $$

Then normalize:

$$ \tilde S[\ell,m] = \frac{S[\ell,m]-\mu_S}{\sigma_S+\epsilon}. $$

This makes every spectrogram approximately zero mean and unit variance.

If recording gain produces approximately a constant additive offset in the log-power representation, per-patch mean subtraction removes that offset.

This can help make a classifier insensitive to microphone gain.

However, it can also remove information about absolute signal level.

Example:

Suppose the task is to distinguish a quiet machine from an excessively loud machine.

If the principal difference is overall power, per-patch normalization may remove much of the desired distinguishing information.

Method 2: Dataset-level normalization

Calculate a shared mean and standard deviation from the training set:

$$ \mu_{\mathrm{train}},\qquad \sigma_{\mathrm{train}}. $$

Then use

$$ \tilde S = \frac{S-\mu_{\mathrm{train}}} {\sigma_{\mathrm{train}}+\epsilon}. $$

The same values must be used for training, validation, testing, and deployment.

This retains more information about how the absolute scale of one recording differs from another.

Method 3: Per-Mel-band normalization

Compute separate statistics for each Mel band:

$$ \mu_\ell = \frac{1}{N_{\mathrm{train}}T} \sum_{i,m}S_i[\ell,m], $$

$$ \sigma_\ell^2 = \frac{1}{N_{\mathrm{train}}T} \sum_{i,m} (S_i[\ell,m]-\mu_\ell)^2. $$

Then

$$ \tilde S_i[\ell,m] = \frac{S_i[\ell,m]-\mu_\ell} {\sigma_\ell+\epsilon}. $$

This compensates for different typical levels across frequency bands.

It is conceptually similar to standardizing each feature in a conventional machine learning pipeline.

The crucial rule is:

Fit normalization statistics using training data only.

Otherwise, information from the validation or test distribution is used during preprocessing.

This is a form of data leakage, even if no class labels are explicitly supplied.

Good student answer: Which normalization is best?

There is no universally best normalization. If absolute loudness matters, shared training-set statistics are often preferable. If recording gain is irrelevant and varies widely, per-patch normalization can improve robustness. Per-Mel-band normalization can compensate for systematic spectral differences between frequency bands. The choice depends on which variations should be preserved and which should be suppressed.

Part V. Tensor shapes, caching, leakage, and reproducibility (Slides 24–30)

Slide 24: Tensor-shape audit

The slide traces the transformation

$$ [C,L] \longrightarrow [1,L] \longrightarrow [1,M,T] \longrightarrow [B,1,M,T]. $$

Every dimension has a different meaning.

Stage	Shape	Interpretation
Loaded waveform	`[C, L]`	Channels, waveform samples
Mono waveform	`[1, L]`	One waveform channel
Mel spectrogram	`[1, M, T]`	One feature channel, Mel bands, time frames
Batch	`[B, 1, M, T]`	Examples, channels, frequency bands, frames
First CNN output	`[B, 16, M, T]`	Sixteen learned feature maps (with shape-preserving convolution)

A common mistake is using

```
features = features.squeeze()
```

This removes every singleton dimension.

If the tensor shape is

```
[1, 64, 297]
```

then `squeeze()` changes it to

```
[64, 297]
```

The `DataLoader` then produces

```
[B, 64, 297]
```

instead of

```
[B, 1, 64, 297]
```

A `Conv2d` layer expects a four-dimensional batched input.

The best repair is to preserve the channel dimension in the Dataset.

For an already constructed batch, the missing dimension can be restored with

```
x = x.unsqueeze(1)
```

This changes

```
[16, 64, 297]
```

into

```
[16, 1, 64, 297]
```

Good student answer: Why is an unconstrained `squeeze()` dangerous?

It removes all dimensions of size one without regard to their meaning. In audio classification, the channel axis may have size one but is still required by the CNN input convention. It is better to retain the axis explicitly or use dimension-specific operations.

Slide 25: On-the-fly features versus cached features

There are two broad ways to prepare spectrograms for model training.

In the first, the Dataset loads a waveform and computes features whenever the example is requested.

In the second, features are computed in advance and stored.

On-the-fly feature computation

Advantages include flexibility and support for waveform augmentation.

Disadvantages include repeated computation and potentially slower data loading.

Cached features

Advantages include faster repeated training and reduced preprocessing cost.

Disadvantages include storage requirements, less flexibility, and the need to manage preprocessing versions.

A useful example is an experiment comparing 64 and 128 Mel bands.

If the features are computed on the fly, changing the number of Mel bands is straightforward.

If features are cached, the cache must be recomputed or separated by configuration.

A sensible cache identifier might include

```
config = {"sample_rate": 16000, "duration": 3.0, "n_fft": 512, "win_length": 512,
          "hop_length": 160, "n_mels": 64, "center": False, "mel_scale": "htk", "power": 2.0,}
```

The configuration can be serialized into a stable cache key.

Good student answer: When should feature caching be used?

Caching is attractive when the feature-extraction parameters are stable and the same spectrograms will be used for many training epochs. On-the-fly computation is preferable during early experimentation or when waveform-domain augmentation must change the features dynamically.

Slide 26: Split recordings before extracting patches

This is one of the most important lessons in the lecture.

Suppose a dataset contains ten recordings from each speaker.

Each recording is divided into many three-second patches.

If patches are randomly divided between training and validation, patches from the same speaker—and possibly the same original recording—may appear in both subsets.

The model can then recognize properties of the speaker, microphone, room, or recording session.

These properties can be highly predictive in the validation set without reflecting the intended classification capability.

This is group leakage.

The proper strategy is to group the data before generating patches.

For speaker-independent speech recognition, split by speaker.

For patient-independent biomedical classification, split by patient.

For generalization to unseen machines, split by machine identity.

For generalization to new sessions from known machines, a session-level split may be appropriate instead.

The correct grouping depends on the intended deployment scenario.

Example: Group-aware splitting

```
from sklearn.model_selection import GroupShuffleSplit

VAL_SIZE = 0.2
RANDOM_SEED = 42

splitter = GroupShuffleSplit(
    n_splits=1,
    test_size=VAL_SIZE,
    random_state=RANDOM_SEED,
)
train_idx, val_idx = next(
    splitter.split(records, labels, groups=speaker_ids)
)
```

All clips from a given speaker are assigned to one subset.

Only after this split should crops or patches be generated.

Good student answer: Why is random patch-level splitting unsafe?

Patches derived from the same original recording or dependent source can share signal content, background noise, and recording artifacts. If those patches appear in both training and validation, the model may exploit those similarities and produce an overly optimistic estimate of generalization.

An important nuance: grouping by speaker prevents speaker overlap, but it does not automatically prevent every other type of leakage. The experimental design must identify the relevant dependencies.

Slide 27: Reproducible data loading

Reproducibility matters because machine learning results depend on both the model and the data pipeline.

Two experiments may use the same architecture and optimizer but differ due to random crops, shuffling, augmentations, or dataset splits.

Useful practices include setting random seeds and recording preprocessing parameters.

```
import random

import numpy as np
import torch

DEFAULT_SEED = 42


def set_seed(seed: int = DEFAULT_SEED) -> None:
    """Sets random seeds across Python, NumPy, and PyTorch for
reproducibility."""
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)


set_seed()
```

For a `DataLoader`, an explicitly seeded generator helps control shuffled ordering:

```
generator = torch.Generator()
generator.manual_seed(42)
```

When using multiple workers, worker-side random number generators may also need controlled initialization.

The essential principle is that a reproducible experiment should be defined by more than the network weights.

It should include the exact data split, preprocessing, augmentation policy, random seeds, software environment, and evaluation procedure.

Good student answer: Why is a fixed validation split not sufficient for reproducibility?

Even if the validation records are unchanged, random crop locations or other stochastic preprocessing can produce different feature tensors on different runs. For stable comparisons, the validation transformation and evaluation procedure must also be deterministic.

Slides 28–29: Notebook walkthrough and in-class activity

The lecture shows this processing sequence:

```
waveform, sr = torchaudio.load(path)
waveform = waveform.mean(0, keepdim=True)
waveform = resample_if_needed(waveform, sr)
waveform, valid = crop_or_pad(waveform, 48_000)
features = to_db(mel(waveform))
features = (features - features.mean()) / features.std().clamp_min(1e-6)
return features, label
```

This is a useful teaching example because it contains nearly every important design decision discussed in the lecture.

Audit question 1: Where does physical standardization happen?

It happens during channel conversion, resampling, and duration standardization.

These operations ensure that the data have consistent physical units and dimensions.

Audit question 2: Where does randomness enter?

Randomness enters during training cropping if the crop start is selected randomly.

Additional randomness might enter through waveform augmentation, shuffling, and stochastic transformations.

Audit question 3: Which information is discarded?

Channel averaging discards spatial differences between channels.

Resampling can discard high-frequency information.

Cropping discards portions outside the selected interval.

Magnitude or power spectrogram calculation discards phase.

Mel filtering reduces frequency detail.

Per-patch normalization removes much absolute scale information.

Audit question 4: What is the shape of the returned tensor?

For the lecture's configuration, it is

```
[1, 64, 297]
```

The `DataLoader` adds the batch dimension.

For eight examples:

```
[8, 1, 64, 297]
```

Why are there 297 time frames?

Given

$$ L=48,000,\qquad N=512,\qquad R=160 $$

and `center=False`, the number of complete STFT frames is

$$ T=1+\left\lfloor\frac{L-N}{R}\right\rfloor. $$

Substituting,

$$ T=1+\left\lfloor\frac{48,000-512}{160}\right\rfloor $$

$$ =1+\lfloor296.8\rfloor $$

$$ =\boxed{297}. $$

This result assumes a 512-sample effective STFT frame length.

Important implementation subtlety: `torchaudio.transforms.MelSpectrogram` uses `center=True` by default. With centered STFT processing and the corresponding padding convention, the frame count is typically

$$ T=1+\left\lfloor\frac{48,000}{160}\right\rfloor=301. $$

Therefore, the lecture's expected 297 frames require explicitly specifying `center=False`.

This is a good debugging exercise: students should be able to explain why their actual tensor contains 301 frames rather than 297.

Here is a shape-check demonstration:

```
import torch
import torchaudio

SAMPLE_RATE = 16_000
N_FFT = 512
WIN_LENGTH = 512
HOP_LENGTH = 160
N_MELS = 64
```

```
waveform = torch.randn(1, 48_000)

mel_transform = torchaudio.transforms.MelSpectrogram(
    sample_rate=SAMPLE_RATE,
    n_fft=N_FFT,
    win_length=WIN_LENGTH,
    hop_length=HOP_LENGTH,
    n_mels=N_MELS,
    center=False,
    power=2.0,
)

mel_features = mel_transform(waveform)
print(mel_features.shape)
```

Expected output:

```
torch.Size([1, 64, 297])
```

The code makes the intended framing convention explicit.

Slide 30: A reliable pipeline makes the model input meaningful

The final slide summarizes the complete idea:

\[\text{Physical audio} \rightarrow \text{DSP representation} \rightarrow \text{Numerical scaling} \rightarrow \text{ML model}. \]

The network's performance depends on every preceding step.

A sophisticated CNN cannot compensate for fundamentally incorrect labels, inconsistent physical units, or leakage between training and validation.

A useful closing question is:

Which pipeline choice could silently leak information about the class label?

Strong answers include:

- Splitting patches from the same source recording across training and validation.
- Allowing recording duration or padding boundaries to correlate with class identity.
- Fitting dataset normalization statistics using validation or test recordings.
- Retaining collection or recording artifacts that uniquely identify classes.
- Performing feature selection or hyperparameter optimization using the final test set.

The best answers do not merely name the problem. They also explain how the model could exploit the unwanted information.

Part VI. Homework problems and expanded solutions (Slides 31–34)

The lecture includes six homework problems. The following solutions extend the slide answers by making the calculations and reasoning explicit.

Problem 1: Frame count and dimensions

Given:

- Recording duration: 2.5 seconds
- Sample rate: 22,050 Hz
- FFT length: 1024
- Hop length: 256
- `center=False`

Find the number of samples, one-sided STFT frequency bins, and complete frames.

Solution

Number of samples:

$$ L=2.5(22,050)=55,125. $$

Number of one-sided frequency bins for a real-valued waveform:

$$ K=\frac{1024}{2}+1=\boxed{513}. $$

Number of frames:

$$ T=1+\left\lfloor\frac{55,125-1024}{256}\right\rfloor $$

$$ =1+\lfloor211.332\ldots\rfloor =\boxed{212}. $$

Therefore, the one-channel spectrogram shape is

`[1, 513, 212]`

If a 64-band Mel transform is applied, the shape becomes

`[1, 64, 212]`

Students should also be able to calculate

$$ \Delta f=\frac{22,050}{1024}\approx21.53 \text{Hz} $$

and

$$ \Delta t=\frac{256}{22,050}\approx11.61 \text{ms}. $$

Problem 2: Representation choice for a 7.2 kHz machine fault

Question: Would you use a linear-frequency or Mel spectrogram?

Suggested answer

I would initially use a linear-frequency spectrogram because the fault is identified by a narrow spectral component around 7.2 kHz. A standard Mel representation compresses high-frequency detail and might combine the fault signature with nearby components.

I would select a sampling rate comfortably above 14.4 kHz and choose an analysis window long enough to resolve the expected frequency differences.

I would then evaluate whether Mel compression improves classification robustness without destroying the narrowband feature.

An especially good graduate-level answer would add that the required window duration depends on the separation between the fault frequency and potential interfering components, as well as the spectral window's main-lobe width and sidelobes.

Problem 3: Training and evaluation crops for rare events

Question: A 30-second recording contains one event lasting one second. Design a crop strategy.

Suggested answer

For training, use event timestamps to construct positive patches guaranteed to contain the event. Also generate negative patches from regions that do not contain the event.

If the event can occur at any point within the three-second training patch, vary its placement as augmentation.

For evaluation, use deterministic sliding windows covering the entire recording and aggregate the window-level predictions.

For rare-event detection, maximum aggregation may be appropriate, although its sensitivity to false positives must be evaluated.

The key distinction is between classifying a patch and classifying an entire recording.

Problem 4: dB calculations

Convert the following power ratios:

$$1,\quad0.25,\quad0.01,\quad10^{-5}.$$

Using

$$P_{\mathrm{dB}}=10\log_{10}(P/P_{\mathrm{ref}}),$$

the answers are:

Ratio	Result
1	0 dB
0.25	−6.02 dB
0.01	−20 dB
\(10^{-5}\)	−50 dB

The additional conceptual answer is that the logarithm is undefined at zero. Power must be clamped or floored before applying the logarithm.

Problem 5: Debug the tensor shape

Given:

```
x.shape == (16, 64, 297)
```

But the CNN expects one input channel.

Problem: The channel dimension is missing.

One possible cause is applying an unconstrained `squeeze()` to individual features before batching.

Preferred repair:

Ensure that each Dataset item has shape

```
[1, 64, 297]
```

Then the DataLoader produces

```
[16, 1, 64, 297]
```

Alternative repair:

```
x = x.unsqueeze(1)
```

Students should also be able to explain why `unsqueeze(1)` is correct: dimension 1 is the channel axis in PyTorch's batched image convention.

Problem 6: Leakage audit for a respiratory dataset

Question: Each patient provides five audio clips. Design a splitting and normalization protocol and give two checks for leakage.

Suggested answer

First, split the dataset by patient identity, not by individual recording. All recordings from one patient must belong to the same subset.

Second, compute shared normalization statistics using only training patients.

Apply those same statistics to validation and test examples.

Two leakage checks are:

1. Verify that the sets of patient IDs in training, validation, and testing have empty pairwise intersections.
2. Check for duplicate or near-duplicate recordings across subsets, including recordings with similar backgrounds, timestamps, or acquisition metadata.

An even stronger answer would discuss whether recording site, microphone type, or hospital identity is correlated with diagnosis and could create a confounding shortcut.

Optional extension: Frame-validity masks

For `center=False`, a window of length \(N\), and a hop of \(R\), the number of frames entirely contained in \(L_{\mathrm{valid}}\) samples is

\[T_{\mathrm{valid}} = \begin{cases} 0, & L_{\mathrm{valid}}<N,\\[4pt] 1+\left\lfloor\frac{L_{\mathrm{valid}}-N}{R}\right\rfloor, & L_{\mathrm{valid}}\geq N. \end{cases} \]

A basic mask is:

```
def make_frame_mask(
    valid_samples,
    total_frames,
    n_fft=512,
    hop_length=160):

    if valid_samples < n_fft:
        valid_frames = 0
    else:
        valid_frames = (1 + (valid_samples - n_fft) // hop_length)
    valid_frames = min(valid_frames, total_frames)
    return (torch.arange(total_frames) < valid_frames)
```

```
import torch

DEFAULT_N_FFT = 512
DEFAULT_HOP_LENGTH = 160


def make_frame_mask(
    valid_samples: int,
    total_frames: int,
    n_fft: int = DEFAULT_N_FFT,
    hop_length: int = DEFAULT_HOP_LENGTH,
) -> torch.Tensor:
    """Creates a boolean mask identifying valid spectrogram frames."""
    if valid_samples < n_fft:
        valid_frames = 0
    else:
        valid_frames = 1 + (valid_samples - n_fft) // hop_length

    valid_frames = min(valid_frames, total_frames)
    return torch.arange(total_frames) < valid_frames
```

This mask marks frames whose complete analysis windows fall within valid original samples. For very short signals, another policy may retain partially valid frames instead, but that choice needs to be explicit.

Part VII. Weekly quiz: Expanded answers (Slides 35–38)

The slide deck includes an eight-question quiz. These are useful for checking whether students understand the DSP-to-ML pipeline rather than merely memorizing software commands.

Question	Good answer
1. What information does an STFT preserve that Welch averaging discards?	The STFT preserves frame-level spectral evolution and temporal order. Welch averages periodograms, thereby discarding which segment contained a particular spectral event.
2. What does a smaller `hop_length` change?	It produces more frames per second, increases overlap and computational redundancy, and decreases spacing between time samples of the STFT. It does not by itself improve the frequency resolution of each window.
3. Why standardize sample rate?	To ensure that frequency bins, frame spacing, and Mel filters have consistent physical meanings across recordings.
4. What shape should one mono Mel patch have?	`[1, n_mels, frames]`. For a batch, `[B, 1, n_mels, frames]`.
5. Power decreases by a factor of 100. What is the change in dB?	-20 dB because $10\log_{10}(1/100)=-20$.
6. Why are random validation crops undesirable?	They may select different signal content on different evaluations, introducing additional variability into accuracy estimates and model comparisons.
7. Give one shortcut created by padding.	The model may use the amount or location of padding to infer the class rather than recognizing the intended acoustic event.
8. One patient contributes clips to	Patient-level splitting failed. The model may exploit patient-specific recording characteristics that are shared across subsets.

Question	Good answer
both train and test. What failed?	

A good grading principle is to award full conceptual credit when students identify both the mechanism and its consequence.

For example, "random cropping introduces noise" is less complete than "random cropping can make the validation input change between runs, causing the measured accuracy to vary even when the network is unchanged."

Part VIII. Additional conceptual questions for senior and graduate students

I would supplement the lecture with the following three questions. They connect the detailed implementation back to broader DSP concepts.

Question A: Is a Mel spectrogram still a linear transformation of the signal?

Suggested answer

No, the complete waveform-to-Mel-power operation is nonlinear.

The STFT itself is linear:

$$ \operatorname{STFT}\{a x_1+b x_2\} = aX_1+bX_2. $$

But converting the complex STFT into power introduces a nonlinear operation:

$$ |aX_1+bX_2|^2 \ne |a|^2|X_1|^2+|b|^2|X_2|^2 $$

in general, because of cross terms.

The Mel filter-bank summation is linear with respect to the power spectrum, but the preceding magnitude-squared operation and subsequent logarithm make the overall feature transformation nonlinear.

This is an excellent connection to the students' earlier study of linear systems.

Question B: Is a CNN invariant to time translation?

Suggested answer

A convolution is translation equivariant, not automatically translation invariant.

If an input pattern shifts, the convolutional response generally shifts with it.

Translation invariance requires additional mechanisms, such as pooling or an appropriate global aggregation operation.

Furthermore, exact equivariance can be affected by boundaries, padding, subsampling, and nonlinear processing.

A CNN classifier can learn approximate time-shift invariance when supported by its architecture and training data.

This distinction is worth emphasizing because students often use "invariance" and "equivariance" interchangeably.

Question C: Could a larger `n_fft` make a CNN perform better even without increasing physical frequency resolution?

Suggested answer

Yes.

A larger FFT can sample the windowed DTFT more densely, changing the numerical feature grid presented to the network.

Even though it does not create new independent information when the window length is fixed, the denser representation can affect interpolation, feature alignment, filter-bank weighting, and the convenience of learning certain patterns.

Better classification performance is therefore possible, but it should not be interpreted as a fundamental increase in frequency resolution.

The extra computational cost and the behavior of the downstream Mel transform also matter.

Part IX. Recommended 75-minute teaching emphasis

The slide deck allocates 15 minutes to STFT/Welch, 15 to spectrograms, 20 to the PyTorch pipeline, 15 to patches and dimensions, and 10 to dB/normalization.

I would keep that structure but deliberately emphasize the following ideas during each block.

Time	Focus	Most valuable student takeaway
0–15 min	Global spectra vs. STFT	Know exactly what time information is retained or discarded
15–30 min	Mel representations and CNNs	Understand that representation choices encode assumptions about relevant information
30–50	PyTorch Dataset and DataLoader	Establish a reliable, physically meaningful tensor contract

Time	Focus	Most valuable student takeaway
min		
50–65 min	Crops, padding, and shapes	Recognize that valid labels and data boundaries matter as much as tensor dimensions
65–75 min	dB, normalization, and leakage	Understand how preprocessing can create or remove predictive information

The three concepts I would most want students to retain are:

First, a spectrogram is not simply an image. It is a physically meaningful time-frequency representation. A CNN can operate on it because of its local structure, but operations such as frequency-axis pooling and translation have signal-processing consequences.

Second, preprocessing determines the classification problem. If a Dataset discards phase, compresses frequencies, removes absolute level, or selects only part of a recording, the network cannot recover information that is no longer represented.

Third, strong validation accuracy does not establish a useful classifier unless the data pipeline is valid. Group leakage, invalid patch labels, and shortcuts from padding or recording conditions can undermine an otherwise sophisticated network.

A good culminating discussion question is:

> Suppose a CNN obtains 98% validation accuracy in classifying machinery faults using three-second Mel spectrograms. What would you want to know before trusting the result?

A strong answer would discuss whether the train/validation split is independent by machine or recording session, whether faults are actually present in the selected patches, whether preprocessing and normalization were fitted correctly, whether nuisance factors such as microphone placement or padding correlate with the labels, and whether evaluation uses unseen machines or operating conditions.

This question integrates the DSP theory, PyTorch architecture, and experimental methodology taught throughout Lecture 27. It also sets up Lecture 28, where the focus shifts from creating the input representation to training and evaluating the CNN itself.

Week 14, Lecture 28

Explain in more detail what's going on here and what good answers and/or examples are.

ChatGPT Medium:

Week 14, Lecture 28: CNN Architectures, Training, and Regularization

Detailed lecture explanations, mathematical intuition, PyTorch examples, and suggested student answers

Course: Digital Signal Processing and Machine Learning\ Audience: Senior undergraduate / first-year graduate ECE students\ Duration: 75 minutes\ Lecture: Week 14, Lecture 28

Lecture 27 introduced the transformation of audio waveforms into log-Mel spectrograms suitable for machine learning. Lecture 28 addresses the next question:

Given a time-frequency representation of an audio signal, how do we design, train, and evaluate a convolutional neural network that learns meaningful acoustic features rather than memorizing the training data?

The lecture has four major parts:

1. CNN architecture: How convolution kernels recognize local time-frequency patterns and how pooling changes the information available to the classifier.
2. Training: How logits, cross-entropy, gradient backpropagation, and optimizers work together to adjust network parameters.
3. Overfitting and regularization: How to diagnose poor generalization and use dropout, weight decay, and early stopping appropriately.
4. SpecAugment: How time and frequency masking encourage a classifier to recognize sounds from incomplete spectrogram information.

An important theme is that these are not independent decisions. Feature extraction, network architecture, the training loop, and the validation procedure must be designed together.

I'll follow the uploaded slide deck, expanding the mathematical details, explaining the reasoning behind the PyTorch code, and providing examples of strong student answers. I will also point out several subtle distinctions that are particularly useful at the senior/graduate ECE level.

Part I. CNN architectures for audio (Slides 3–14)

Slide 3: Opening diagnostic — 99% training accuracy, 58% validation accuracy

The lecture opens with a common machine learning problem:

A CNN achieves 99% accuracy on training recordings but only 58% accuracy on validation recordings. Furthermore, validation loss begins increasing after epoch 7.

The question is what could explain these observations.

The slides propose three hypotheses:

- The model is memorizing training-specific details.
- The validation transformation is stochastic or inconsistent.
- The data split has source-related leakage or other grouping problems.

These are different problems and require different remedies.

Hypothesis A: The model is overfitting

The model may have enough parameters to learn recording-specific features that do not generalize.

For example, suppose we train a CNN to identify a defective motor from its spectrogram.

The training recordings might have been collected using a particular microphone in a particular room.

The network could learn the microphone's frequency response, the room's reverberation, or the background noise instead of the motor fault.

If the validation set contains recordings collected under different conditions, accuracy can be substantially lower.

A characteristic learning curve is:

- Training loss steadily decreases.
- Validation loss initially decreases.
- Validation loss later increases.

This indicates that additional training improves the fit to training examples without improving predictive performance on validation examples.

Hypothesis B: Validation is not deterministic

Suppose the validation Dataset uses random three-second crops from recordings that are ten seconds long.

A clip might contain the relevant event in one crop but not another.

The validation metric can then fluctuate even when the network parameters have not changed.

Similarly, leaving dropout active during validation introduces randomness.

This is a problem with measurement reliability, not necessarily a problem with model capacity.

Hypothesis C: The dataset split is invalid

If patches from the same original recording appear in both training and validation, the model can exploit shared recording characteristics.

However, ordinary leakage typically makes validation performance artificially optimistic, rather than directly explaining a very low validation accuracy.

A low validation accuracy can instead result from a poorly designed split, severe distribution differences, or inconsistent labels.

That distinction is worth making explicit for students.

Good student answer

> The large training-validation gap suggests poor generalization, but it does not uniquely prove overfitting. I would first verify that validation is deterministic, that the model is in evaluation mode, and that the data split respects recording or source groups. I would then inspect learning curves and compare against a smaller model or stronger regularization. If validation performance is reproducible and the split is valid, a steadily decreasing training loss combined with increasing validation loss is strong evidence of overfitting.

Suggested follow-up: If the model gets 99% on both training and validation, can we conclude it generalizes?

No. If the data split is contaminated, both scores can be misleading. An independent test set with an appropriate grouping protocol is needed.

Slide 4: The input contract — batch, channel, Mel, time

The CNN receives a tensor

\[X\in\mathbb{R}^{B\times1\times M\times T} \]

where:

- \(B\): mini-batch size
- \(1\): number of input channels
- \(M\): number of Mel-frequency bands
- \(T\): number of time frames

For example,

```
x.shape = [8, 1, 64, 297]
```

means that eight spectrograms are being processed simultaneously.

Each spectrogram contains 64 Mel-frequency bands and 297 STFT frames.

In PyTorch, `Conv2d` expects

```
[batch, channels, height, width]
```

For an audio spectrogram, height corresponds to Mel frequency and width corresponds to time.

This correspondence is important because CNN operations are defined in terms of array dimensions, but we must interpret those operations using the physical meaning of the signal.

Example

```
import torchfrom torch import nnx = torch.randn(8, 1, 64, 297)conv =
nn.Conv2d(    in_channels=1,    out_channels=16,    kernel_size=3,
padding=1)y = conv(x)print(y.shape)
```

Output:

```
torch.Size([8, 16, 64, 297])
```

The convolution creates 16 new feature maps from the original single-channel spectrogram.

Each feature map represents the response of one learned convolutional filter.

Good student answer: What does the channel dimension mean before and after convolution?

Before the first convolution, the channel dimension identifies the single log-Mel input representation. After the convolution, the 16 channels represent 16 learned feature maps. Each map detects a different pattern in the time-frequency input.

Slide 5: Conv2d learns local spectro-temporal detectors

This slide establishes the connection between CNNs and signal processing.

A spectrogram contains recognizable geometric patterns:

- Horizontal ridges represent persistent frequencies.
- Vertical stripes represent short, broadband events.
- Diagonal ridges represent frequency sweeps.
- Distributed textures represent broadband or noise-like signals.

Steady tone

Horizontal ridge

Impulse or onset

Vertical stripe

Rising chirp

Diagonal ridge

Broadband noise

Diffuse texture

Illustrative spectrogram patterns. A trained CNN can learn local filters responding to parts of these structures.

The convolution operation

Ignoring padding conventions, a two-dimensional convolutional layer computes a weighted sum over a local input region.

For output channel o,

$$Y_o[i,j] = b_o+ \sum_{c=0}^{C_{\mathrm{in}}-1} \sum_{u=0}^{K_h-1} \sum_{v=0}^{K_w-1} W_{o,c}[u,v]X_c[i+u,j+v].$$

This expression is technically the cross-correlation operation implemented by most CNN libraries, because the kernel is not reversed.

In traditional DSP notation, convolution involves a reversal of the impulse response.

That difference generally does not matter for CNN training because the kernel coefficients are learned rather than predetermined.

It is nevertheless a worthwhile point for DSP students.

Example: A detector for changes in time

Consider a simple 3×3 kernel:

$$W= \begin{bmatrix} -1&0&1\\ -1&0&1\\ -1&0&1 \end{bmatrix}.$$

The kernel compares spectrogram intensity on the left with intensity on the right.

It produces a strong response when energy changes rapidly across time.

Such a pattern can help detect a sudden onset.

A complementary kernel,

$$W= \begin{bmatrix} -1&-1&-1\\ 0&0&0\\ 1&1&1 \end{bmatrix},$$

detects changes in the frequency direction.

A CNN learns its own kernels from data rather than requiring these to be designed by hand.

Good student answer: What can a 3×3 kernel detect?

A 3×3 kernel can detect local time-frequency patterns such as a small segment of a horizontal ridge, a temporal onset, a local frequency edge, or part of a diagonal chirp. Multiple kernels learn different patterns, and deeper layers combine these local features into larger acoustic structures.

Important connection to LTI systems

A convolutional layer resembles a bank of two-dimensional FIR filters.

However, a complete CNN is not an LTI system because it typically includes nonlinearities, pooling, normalization, and finite-boundary operations.

A convolution followed by ReLU is nonlinear:

$$Y=\max(0,W*X+b).$$

This nonlinear feature extraction allows a CNN to distinguish complex combinations of acoustic patterns that a single linear filter cannot.

Slide 6: Kernel geometry encodes an inductive bias

The slide compares four kernel types.

Kernel	Neighborhood	Example acoustic feature
3×3	Small frequency and time region	Local edge or blob
3×9	Narrow in Mel, longer in time	Persistent tone or ridge
9×3	Wider in Mel, shorter in time	Broadband onset
Dilated 3×3	Sparse, enlarged neighborhood	Longer-range modulation

An inductive bias is a structural assumption built into a learning algorithm.

For CNNs, one such assumption is that nearby input values are related and useful patterns can be recognized with shared filters.

The important issue for audio is that the horizontal and vertical axes have different physical meanings.

A 3×9 kernel covers three Mel bands and nine time frames.

If the hop length is 160 samples and the sample rate is 16 kHz, the time spacing between the first and last kernel positions is

$$(9-1)\frac{160}{16,000} = 0.08 \text{s}.$$

Thus, the kernel spans 80 ms between the centers of its first and last frames.

If each STFT frame also uses a 32 ms analysis window, the waveform support involved is approximately

$$80+32=112 \text{ms}.$$

This distinction between frame-center span and underlying waveform support is important.

Why choose a 3×9 kernel?

A relatively long time kernel can integrate a narrowband pattern across many frames.

It may be useful for identifying stable tones or short harmonic structures.

Why choose a 9×3 kernel?

A frequency-tall but time-short kernel can integrate energy across a broader range of Mel bands over a brief time interval.

This is useful for recognizing transients or broadband onsets.

What does dilation do?

A dilated convolution spreads kernel samples apart.

For a kernel of width k and dilation d, the effective receptive-field width is

$$K_{\mathrm{eff}}=d(k-1)+1.$$

For $k=3$ and $d=2$,

$$K_{\mathrm{eff}}=2(3-1)+1=5.$$

Thus, a 3×3 kernel with dilation 2 samples positions over a 5×5 region without increasing the number of learned kernel coefficients.

Good student answer: Why might anisotropic kernels be better for audio?

Because time and frequency are physically different axes. We may want a detector that integrates information over a long time interval but a narrow frequency band, or one that captures a broadband event over a short time. Rectangular kernels can encode these task-dependent assumptions.

Slide 7: Convolution output size

Students should be able to calculate tensor dimensions through a network without relying solely on running code.

For one spatial dimension, the output length is

$$L_{\mathrm{out}} = \left\lfloor \frac{L_{\mathrm{in}}+2p-d(k-1)-1}{s}+1 \right\rfloor$$

where:

- L_{in}: input dimension

- p: padding on each side
- d: dilation
- k: kernel size
- s: stride

Example: Shape-preserving convolution

Given

$$L_{\mathrm{in}}=64,\quad k=3,\quad p=1,\quad s=1,\quad d=1,$$

we have

$$L_{\mathrm{out}} = \left\lfloor\frac{64+2-2-1}{1}+1\right\rfloor =64.$$

Thus, a 3×3 convolution with padding 1 and stride 1 preserves both spatial dimensions.

Now apply 2×2 max pooling with stride 2.

$$64\longrightarrow32.$$

Applying three such pooling layers gives

$$64\longrightarrow32\longrightarrow16\longrightarrow8.$$

For a time dimension of 173 frames:

$$173\longrightarrow86\longrightarrow43\longrightarrow21.$$

Good student answer: What happens when a pooling layer receives an odd dimension?

For a standard 2×2 pooling layer with stride 2 and no padding or ceiling mode, the output is the floor of half the input size. Therefore, 173 becomes 86, not 87.

Slides 8–9: A lightweight CNN and its parameter count

The lecture proposes the following architecture:

CNN signal flow

Input log-Mel spectrogram

[B, 1, 64, T]

Block 1

16 channels

Conv → BN → ReLU → Pool

Block 2

32 channels

Conv → BN → ReLU → Pool

Block 3

64 channels

Conv → BN → ReLU → Pool

Global average pooling

[B, 64, 1, 1]

Flatten → Dropout → Linear

Output: [B, C] class logits

Three properties are worth emphasizing.

First, the number of channels increases:

$$ 1\rightarrow16\rightarrow32\rightarrow64. $$

This gives later layers more learned feature representations.

Second, the spatial dimensions decrease because of pooling.

Third, global average pooling reduces the output to 64 features, regardless of the time dimension.

Counting convolutional parameters

For a convolutional layer,

$$ N_{\mathrm{weights}} = C_{\mathrm{out}}C_{\mathrm{in}}K_hK_w. $$

If bias is enabled, add C_{out}.

For `Conv2d(1,16,3,bias=False)`,

$$ N=16(1)(3)(3)=144. $$

For `Conv2d(16,32,3,bias=False)`,

$$ N=32(16)(3)(3)=4608. $$

For `Conv2d(32,64,3,bias=False)`,

$$ N=64(32)(3)(3)=18,432. $$

A linear classifier with 64 inputs and 10 classes requires

$$ N_{\mathrm{linear}}=10(64)+10=650 $$

parameters, including bias.

Notice the striking difference between this head and one that flattens all spatial features.

After three pooling operations, a 64×128 input becomes 64×8×16.

Flattening gives

$$64(8)(16)=8192$$

features.

A ten-class linear head would then require

$$8192(10)+10=81,930$$

parameters.

Compare:

Global-pooling head

650

Trainable parameters

Flattening head

81,930

Trainable parameters

The flattening head has approximately 126 times as many parameters.

This does not prove that global pooling is always better, but it substantially reduces head capacity and often lowers the risk of memorizing training-specific patterns.

Good student answer: Why use global average pooling before the classifier?

Global average pooling produces a fixed-size feature vector independent of spectrogram dimensions. It reduces the number of classifier parameters, can improve robustness to shifts in event location, and avoids constructing a large fully connected layer from flattened feature maps. The tradeoff is that detailed spatial information is lost.

Slide 10: Pooling trades localization for invariance

Pooling reduces the spatial dimensions of a feature map.

The lecture contrasts max pooling, average pooling, and adaptive average pooling.

Max pooling

For a region $\mathcal{R}$,

$$y=\max_{(i,j)\in\mathcal{R}}x[i,j].$$

Max pooling retains the largest activation.

This can be useful when a class is characterized by the presence of a particular event.

For example, if a spectrogram contains one brief siren, a strongly activated feature detector may be sufficient to indicate the event's presence.

Average pooling

Average pooling computes

$$y=\frac{1}{|\mathcal{R}|} \sum_{(i,j)\in\mathcal{R}}x[i,j].$$

It measures the average activity over the region.

This can be useful for distributed patterns such as sustained broadband machine noise.

Why does pooling affect invariance?

A small translation of an input feature may move its activation within a pooling region without greatly changing the pooled output.

Consequently, the network becomes less sensitive to exact feature position.

But this is not necessarily desirable in both spectrogram dimensions.

In time, event position may be irrelevant.

In frequency, event position may be essential.

For example, a 500 Hz machine vibration and a 1 kHz vibration may indicate different operating conditions.

Pooling that merges their feature responses could reduce the ability to distinguish them.

Good student answer: Is pooling always beneficial?

No. Pooling can reduce computation, increase robustness to small shifts, and help summarize features. But it discards localization information. If the classification task depends on precise frequency or temporal relationships, aggressive pooling may reduce performance.

Slide 11: AdaptiveAvgPool2d and variable time width

A major advantage of adaptive average pooling is that it produces a user-specified output shape.

For example:

```
pool = nn.AdaptiveAvgPool2d((1, 1))
```

maps

```
[B, 64, 8, 12]
```

and

```
[B, 64, 8, 25]
```

to

```
[B, 64, 1, 1]
```

The classifier therefore always receives 64 features.

This is extremely useful when recordings have different durations and consequently different spectrogram time dimensions.

However, it is important to distinguish two ideas:

- The model can accept different time widths in different forward passes.
- A standard `DataLoader` still needs compatible tensor shapes to stack examples into a single batch.

Adaptive pooling solves the first problem, not automatically the second.

Variable-length examples may still require padding, bucketing, or custom batching.

A potential disadvantage

Global pooling averages across both time and frequency.

Suppose two signals contain the same spectral events in opposite temporal order.

A model using fully global pooling may struggle to preserve that ordering, especially when the preceding feature maps do not already encode the relevant relationships.

A possible alternative is

```
nn.AdaptiveAvgPool2d((4, 1))
```

or another non-global spatial output shape, chosen according to which dimension should retain coarse location information.

For example, `(1,4)` preserves four coarse time regions while averaging across the frequency dimension; `(4,1)` preserves four coarse frequency regions while averaging across time.

Good student answer: When is global pooling inappropriate?

Global pooling may be inappropriate when the label depends on precise frequency location or temporal ordering. It summarizes the presence and average strength of learned features but discards much spatial information. A classifier for sequences of bird calls, for instance, may need to preserve more temporal structure than a simple sound-presence classifier.

Slides 12–13: Complete CNN architecture and shape audit

The lecture uses a compact three-block model.

Here is a runnable implementation consistent with the intended architecture.

```
import torch
from torch import nn

class Block(nn.Module):
    def __init__(self, in_channels, out_channels):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(
                in_channels,
                out_channels,
                kernel_size=3,
                padding=1,
                bias=False
            ),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )
    def forward(self, x):
        return self.net(x)

class AudioCNN(nn.Module):
    def __init__(self, n_classes, p=0.3):
        super().__init__()
        self.features = nn.Sequential(
            Block(1, 16), Block(16, 32), Block(32, 64))
        self.pool = nn.AdaptiveAvgPool2d((1, 1))
        self.head = nn.Sequential(
            nn.Flatten(), nn.Dropout(p), nn.Linear(64, n_classes))
    def forward(self, x):
        return self.head(self.pool(self.features(x)))  # logits
```

```
import torch
from torch import nn


class ConvBlock(nn.Module):
    """A standard Conv2d -> BatchNorm -> ReLU -> MaxPool building block."""

    def __init__(self, in_channels: int, out_channels: int) -> None:
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(
                in_channels,
                out_channels,
                kernel_size=3,
                padding=1,
                bias=False,
            ),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.net(x)


class AudioCNN(nn.Module):
    """Convolutional neural network for audio spectrogram classification."""

    def __init__(self, num_classes: int, dropout_prob: float = 0.3) -> None:
        super().__init__()
        self.features = nn.Sequential(
            ConvBlock(1, 16),
            ConvBlock(16, 32),
            ConvBlock(32, 64),
        )
        self.pool = nn.AdaptiveAvgPool2d((1, 1))
```

For an input of shape

`[8, 1, 64, 173]`

the dimensions are:

Layer	Output shape
Input	`[8, 1, 64, 173]`
Conv 1	`[8, 16, 64, 173]`
MaxPool 1	`[8, 16, 32, 86]`
Conv 2	`[8, 32, 32, 86]`
MaxPool 2	`[8, 32, 16, 43]`
Conv 3	`[8, 64, 16, 43]`
MaxPool 3	`[8, 64, 8, 21]`

Layer	Output shape
Adaptive average pooling	`[8, 64, 1, 1]`
Flatten	`[8, 64]`
Linear (10 classes)	`[8, 10]`

This shape audit is an excellent classroom activity because it combines CNN design with the dimensional reasoning already familiar from DSP.

Slide 14: Design checkpoint — What should remain after pooling?

The slide presents three classification tasks.

Siren detection: The model may mainly need to determine whether a siren occurs anywhere in a recording.

Global pooling may be reasonable because exact onset time is not necessarily important.

Bird-call sequence recognition: The order in which acoustic events occur may help identify the species.

A model that retains temporal information, perhaps with temporal convolutions or an attention mechanism, may be more suitable.

Machine-fault identification: Particular frequencies can have precise physical significance.

Aggressive frequency pooling may remove useful distinctions, so retaining some absolute frequency structure can be valuable.

The essential concept is:

Pooling is a statement about which information the classifier is allowed to ignore.

A good student answer should justify pooling relative to the classification task, not simply say that pooling reduces the number of parameters.

Part II. The training loop (Slides 15–24)

Slide 16: Logits, targets, and CrossEntropyLoss

After the CNN processes a spectrogram, its final linear layer produces a vector of class scores called logits.

Suppose there are three classes:

- Siren
- Engine
- Dog bark

For one recording, the network might produce

$$ \mathbf z= \begin{bmatrix} 2.0\ 0.5\ -1.0 \end{bmatrix}. $$

These values are not probabilities.

They can be positive or negative, and they do not have to sum to one.

To interpret them as class probabilities, we use the softmax function:

$$ p_c = \frac{e^{z_c}}{\sum_{j=1}^{C}e^{z_j}}. $$

For the example,

$$ e^2\approx7.389, \qquad e^{0.5}\approx1.649, \qquad e^{-1}\approx0.368. $$

The sum is approximately 9.406.

Therefore,

$$ \mathbf p\approx \begin{bmatrix} 0.786\ 0.175\ 0.039 \end{bmatrix}. $$

The classifier assigns its largest probability to the siren class.

How does cross-entropy measure classification error?

If the correct class is y, the cross-entropy loss is

$$ \ell=-\log p_y. $$

For a batch containing B examples,

$$ \mathcal L = -\frac1B\sum_{i=1}^{B}\log p_{i,y_i}. $$

This loss penalizes the network when it assigns a low probability to the correct class.

If the correct class is the siren class in the example above,

$$ \ell=-\log(0.786)\approx0.241. $$

If the correct class were dog bark instead,

$$ \ell=-\log(0.039)\approx3.24. $$

Thus, the network receives a much larger penalty for confidently favoring the wrong class.

Why doesn't the PyTorch model apply softmax?

`nn.CrossEntropyLoss` expects raw logits and internally combines log-softmax with negative log-likelihood.

This is numerically more stable than computing probabilities separately and then taking their logarithms.

The standard PyTorch usage is:

```
loss_fn = nn.CrossEntropyLoss()
logits = model(x)
loss = loss_fn(logits, y)
```

where

```
logits.shape == [B, C]
y.shape == [B]
y.dtype == torch.long
```

For a three-class problem, the target might be

```
y = torch.tensor([0, 2, 1, 0])
```

Students should understand that class labels here are integer indices, not numerical measurements.

Good student answer: Why should we not put softmax at the end of the model?

Because `CrossEntropyLoss` expects logits and performs the required log-softmax computation internally. Applying softmax before the loss is redundant and changes the optimization behavior. It can also reduce numerical stability.

An additional nuance: PyTorch cross-entropy can support correctly formed soft-label probability targets. However, the lecture's intended task is standard single-label multiclass classification, where integer class IDs are the simplest and preferred interface.

Slide 17: One training step — The order matters

This is one of the most important slides in the lecture.

A complete training step consists of five operations:

1. Zero the previous gradients.
2. Compute predictions using the model.
3. Evaluate the loss.
4. Compute gradients through backpropagation.
5. Update the parameters using the optimizer.

A typical PyTorch implementation is:

```
optimizer.zero_grad(set_to_none=True)
logits = model(x)
loss = loss_fn(logits, y)
loss.backward()
optimizer.step()
```

Let's connect each operation to the underlying mathematics.

Step 1: Zero gradients

```
optimizer.zero_grad(set_to_none=True)
```

PyTorch accumulates gradients in each parameter's `.grad` attribute by default.

If gradients are not cleared, the gradients from the current mini-batch will be added to those from previous mini-batches.

There are legitimate uses for gradient accumulation, but it should be intentional.

Step 2: Forward propagation

```
logits = model(x)
```

Let the entire CNN represent the parameterized nonlinear function

$$ \mathbf z=f_\theta(\mathbf x) $$

where θ contains all trainable convolutional and classifier weights.

The forward pass calculates the logits from the current parameters.

Step 3: Compute loss

```
loss = loss_fn(logits, y)
```

The loss measures disagreement between the predictions and target classes.

For B examples, it is

$$ \mathcal L(\theta) = -\frac1B \sum_{i=1}^{B} \log p_{i,y_i}(\theta). $$

Step 4: Backpropagation

```
loss.backward()
```

This calculates gradients such as

$$ \frac{\partial\mathcal L}{\partial W} $$

for every trainable parameter tensor.

The gradients are calculated using the chain rule through the computational graph.

This is closely related to the adaptive filter gradient calculations encountered earlier in the DSP course.

Step 5: Parameter update

```
optimizer.step()
```

For basic gradient descent,

$$ \theta_{t+1} = \theta_t-\eta\nabla_\theta\mathcal L(\theta_t) $$

where η is the learning rate.

More sophisticated optimizers such as AdamW use additional information derived from previous gradients, but the objective remains reducing the loss.

Useful mathematical extension: Gradient of softmax cross-entropy

For one example, let

$$ \ell=-\log p_y $$

with

$$ p_c=\frac{e^{z_c}}{\sum_j e^{z_j}}. $$

The derivative of the loss with respect to logit z_c is

$$ \boxed{ \frac{\partial\ell}{\partial z_c} = p_c-\mathbb{1}[c=y] } $$

where $\mathbb{1}[c=y]$ is 1 for the correct class and 0 otherwise.

This is an especially useful result for understanding backpropagation.

If the correct class is $y=0$, the gradient with respect to its logit is

$$ p_0-1. $$

If the correct class probability is low, this value is strongly negative. Gradient descent therefore tends to increase that logit.

For incorrect classes, the gradient is $p_c>0$, so gradient descent tends to decrease their logits.

Good student answer: Why does backpropagation reduce classification errors?

The loss gradient tells us how each model parameter affects the classification loss locally. The optimizer changes parameters in directions intended to reduce that loss. For softmax cross-entropy, the gradient encourages the correct-class logit to increase relative to incorrect-class logits.

Slide 18: A robust training and validation epoch function

The lecture presents a useful design pattern: use the same function for both training and validation, while controlling whether parameter updates are enabled.

A more complete implementation is:

```
def run_epoch(model, loader, loss_fn, device, optimizer=None):
    training = optimizer is not None
    model.train(training)
    total_loss = 0.0    total_correct = 0    total_examples = 0
    context = (torch.enable_grad() if training else torch.inference_mode())
    with context:
        for x, y in loader:
            x = x.to(device)     y = y.to(device)
            if training: optimizer.zero_grad(set_to_none=True)
            logits = model(x)
            loss = loss_fn(logits, y)
            if training:
                loss.backward()
                optimizer.step()
                batch_size = y.size(0)
                total_loss += loss.item() * batch_size
```

This assumes the default mean reduction for `CrossEntropyLoss`.

Why multiply `loss.item()` by batch size?

Suppose the DataLoader produces three batches:

- Batch 1: 32 examples
- Batch 2: 32 examples
- Batch 3: 6 examples

Each batch loss is a mean over its examples.

If we simply average the three batch losses, the six-example batch receives the same weight as a 32-example batch.

That is not the desired average loss over all 70 examples.

Instead, compute

$$ \mathcal L_{\mathrm{epoch}} = \frac{\sum_b B_b\mathcal L_b}{\sum_b B_b}. $$

This ensures that each example contributes equally to the reported average.

Good student answer: Why can't we simply average the loss from every batch?

Because batches may have different sizes, particularly the final batch. An unweighted average of batch means gives smaller batches disproportionate influence. Multiplying each batch mean by its batch size before averaging over all examples produces the correct sample-weighted mean.

If class weights or nonstandard reduction are used, the aggregation may require a different denominator. The principle remains that the reported metric must match the definition of the loss being measured.

Slide 19: Training mode versus evaluation mode

PyTorch distinguishes between

```
model.train()
```

and

```
model.eval()
```

These methods control the behavior of certain network layers.

They do not directly enable or disable automatic differentiation.

The most important examples are dropout and batch normalization.

Dropout

During training, dropout randomly sets some activations to zero.

During evaluation, dropout is disabled.

This makes validation predictions deterministic with respect to dropout.

Batch normalization

Batch normalization uses statistics of the current mini-batch during training and normally maintains running estimates of mean and variance.

During evaluation, the default configuration uses these stored estimates.

If validation is accidentally performed in training mode, the BatchNorm running statistics can be updated from validation data, and predictions may depend on the composition of validation batches.

This contaminates the evaluation procedure.

Gradient computation

Calling `model.eval()` does not disable gradients.

For validation, use

```
model.eval()with torch.inference_mode():
logits = model(x)
```

The inference context avoids building the autograd graph and can reduce memory use and computational overhead.

It should not be used during training because the backward pass needs the computational graph.

Good student answer: What does `model.eval()` do, and what does it not do?

It changes the behavior of modules such as dropout and batch normalization to their evaluation behavior. It does not automatically disable gradient tracking. We should also use `torch.inference_mode()` or `torch.no_grad()` when performing validation without gradients.

Slide 20: AdamW versus SGD with momentum

The slide compares two common optimizers.

Stochastic gradient descent

A basic SGD update is

$$ \theta_{t+1}=\theta_t-\eta g_t $$

where

$$ g_t=\nabla_\theta\mathcal L(\theta_t). $$

With momentum, the optimizer maintains a state that incorporates previous gradients.

One common formulation is

$$ v_t=\beta v_{t-1}+g_t $$

followed by

$$ \theta_{t+1}=\theta_t-\eta v_t. $$

Momentum smooths the direction of parameter updates and can accelerate optimization along directions with persistent gradients.

Adam

Adam maintains estimates of the first and second moments of the gradient:

$$ m_t=\beta_1m_{t-1}+(1-\beta_1)g_t $$

$$ v_t=\beta_2v_{t-1}+(1-\beta_2)g_t^2. $$

After correcting initialization bias,

$$ \hat m_t=\frac{m_t}{1-\beta_1^t} $$

$$ \hat v_t=\frac{v_t}{1-\beta_2^t}. $$

The adaptive update is approximately

$$ \theta_{t+1} = \theta_t- \eta\frac{\hat m_t}{\sqrt{\hat v_t}+\epsilon}. $$

The division and square root are elementwise.

This means different parameters can effectively receive different update scales based on their gradient history.

AdamW modifies this by using decoupled weight decay.

Why use AdamW for the lecture baseline?

AdamW is often a practical default for training a relatively small CNN because it can make rapid initial progress without requiring the same amount of learning-rate tuning as some SGD configurations.

However, it is not guaranteed to provide better final generalization.

A fair optimizer comparison requires consistent data splits, augmentation, model architecture, metric definitions, and computational budget.

Good student answer: Why might AdamW converge faster initially than SGD?

AdamW adapts parameter updates using estimates of gradient magnitude and history. This can improve optimization when gradients have different scales across parameters. SGD relies more directly on a shared learning rate and momentum, so it may require more careful tuning.

Slide 21: Weight decay constrains parameter growth

Weight decay is a regularization mechanism that discourages excessively large parameter values.

For basic gradient descent with L2 regularization, consider the objective

$$ \mathcal J(\theta) = \mathcal L(\theta) + \frac{\lambda}{2}\|\theta\|_2^2. $$

Its gradient is

$$ \nabla_\theta\mathcal J = \nabla_\theta\mathcal L+\lambda\theta. $$

The update becomes

$$ \theta_{t+1} = \theta_t- \eta\left(\nabla_\theta\mathcal L+\lambda\theta_t \right). $$

Rearranging,

$$ \theta_{t+1} = (1-\eta\lambda)\theta_t -\eta\nabla_\theta\mathcal L. $$

The term

$$ (1-\eta\lambda)\theta_t $$

shrinks parameter values toward zero.

Why is AdamW different?

With adaptive optimizers such as Adam, including an L2 penalty in the loss gradient is not equivalent to applying a separate parameter shrinkage.

AdamW explicitly separates weight decay from the adaptive gradient step.

A simplified representation is

$$\boxed{ \theta_{t+1} = (1-\eta\lambda)\theta_t - \eta\frac{\hat m_t}{\sqrt{\hat v_t}+\epsilon} }$$

where the moment estimates are formed from the loss gradients rather than from gradients that include an L2 penalty.

What does weight decay accomplish?

It discourages the optimizer from relying on unnecessarily large parameter coefficients.

This can improve generalization, although it does not guarantee it.

Too much weight decay may prevent the model from fitting even the training data.

This produces underfitting.

The slide suggests testing log-spaced values such as

$$10^{-5},\quad10^{-4},\quad10^{-3},\quad10^{-2}.$$

These are reasonable experimental candidates, not universal optimal values.

Good student answer: Why is increasing weight decay not always beneficial?

A moderate amount can reduce overfitting by limiting parameter growth. Too much weight decay makes it difficult for the model to represent useful patterns, increasing training loss and potentially causing underfitting. It should be tuned jointly with the learning rate using validation results.

Slide 22: Cosine annealing changes the learning rate smoothly

A learning-rate scheduler changes the optimizer's step size during training.

Cosine annealing uses a schedule of the form

$$\eta_t = \eta_{\min} + \frac12(\eta_{\max}-\eta_{\min}) \left[1+\cos\left(\frac{\pi t}{T}\right) \right].$$

Here:

- $\eta_{\max}$: initial or maximum learning rate
- $\eta_{\min}$: minimum learning rate
- T: duration of the schedule
- t: current schedule step

Example cosine learning-rate schedule

Smooth reduction from 0.001 to 0.00001 over 20 epochs

00.00030.00060.00090.0012024681012141618 20

During early training, relatively large steps allow the optimizer to make substantial parameter changes.

Later, smaller steps allow finer adjustment.

A typical implementation is

```
import torch

LEARNING_RATE = 1e-3
WEIGHT_DECAY = 1e-3
NUM_EPOCHS = 20
MIN_LEARNING_RATE = 1e-5

optimizer = torch.optim.AdamW(
    model.parameters(),
    lr=LEARNING_RATE,
    weight_decay=WEIGHT_DECAY,
)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
    optimizer,
    T_max=NUM_EPOCHS,
    eta_min=MIN_LEARNING_RATE,
)
```

If `T_max` is measured in epochs, call

```
scheduler.step()
```

once per epoch.

If instead the scheduler is intended to operate at mini-batch resolution, its duration and stepping frequency must be defined accordingly.

Good student answer: What goes wrong if the scheduler is stepped at the wrong frequency?

The learning rate changes on a different time scale than intended. For example, a schedule designed to last 20 epochs may complete after only 20 mini-batches if the scheduler is stepped after every batch. This can make the learning rate decrease far too quickly.

Slide 23: Validation is an experiment, not a progress bar

Validation is used to estimate how well a trained model performs on data not used for fitting its parameters.

The key is that validation must be a controlled experiment.

Several practices are required:

- Use a fixed validation split.
- Keep validation feature extraction deterministic.
- Do not apply random training augmentations to validation examples.
- Aggregate metrics consistently across the dataset.
- Select checkpoints using a predefined validation criterion.
- Reserve the test data for final evaluation after model selection.

Why shouldn't we optimize directly on test accuracy?

Suppose a student trains 50 models and selects the one with the best test accuracy.

Even if no gradient was computed on the test set, that test set has influenced model selection.

The final test result is no longer an unbiased evaluation of a model selected independently of the test data.

Instead, use validation data for selecting hyperparameters and checkpoints.

Then evaluate the selected model on the test set.

What metrics should be reported?

Accuracy may be sufficient for balanced multiclass problems, but it can be misleading when classes are imbalanced.

For example, suppose 95% of recordings are healthy and 5% contain a fault.

A classifier that always predicts healthy achieves 95% accuracy but detects no faults.

For such problems, students should consider precision, recall, F1 score, confusion matrices, and possibly class-specific detection measures.

Good student answer: Why can high accuracy be misleading?

Accuracy measures the total fraction of correct predictions but does not reveal how errors are distributed among classes. In an imbalanced dataset, the classifier can achieve high accuracy by favoring the majority class while performing poorly on rare but important events.

Slide 24: Training-loop failure modes

The lecture identifies several common symptoms.

Symptom	Possible cause	Useful diagnostic
Training loss never decreases	Incorrect target encoding, unsuitable learning rate, broken gradient flow	Check dtypes, loss, gradients, and optimizer
Validation results	Random evaluation transforms or	Repeat validation in

Symptom	Possible cause	Useful diagnostic
fluctuate unexpectedly	dropout active	evaluation mode
Accuracy high but loss high	Confident errors on a subset of examples	Inspect logits and class probabilities
Learning-rate schedule appears ineffective	Wrong stepping frequency	Log learning rate during training

One of the most valuable debugging techniques in the slide is the tiny-batch overfit test.

What is a tiny-batch overfit test?

Take one or two small batches of correctly labeled examples and train repeatedly on only those examples.

With a sufficiently expressive network and an appropriate optimizer, the model should often be able to fit this tiny dataset very closely.

If it cannot, there may be an implementation error.

Potential causes include:

- Wrong input or target shapes.
- Missing `loss.backward()`.
- Missing `optimizer.step()`.
- Very small or very large learning rate.
- Incorrect labels.
- Excessive dropout or augmentation during the test.
- Unintentionally frozen parameters.

For debugging, I would temporarily disable augmentation and dropout, while leaving the rest of the architecture intact, to make the experiment easier to interpret.

Good student answer: Why should we try to overfit one batch before tuning regularization?

If the model cannot fit a tiny dataset, the problem is likely related to implementation, optimization, insufficient capacity, or incorrect data rather than excessive memorization. The tiny-batch test verifies that the forward pass, loss, gradients, and parameter updates can work together.

Part III. Overfitting and dropout (Slides 25–29)

Slide 26: Learning curves reveal different failures

A model's training and validation losses provide useful diagnostic information.

The slide distinguishes three patterns.

Illustrative learning curves

Healthy fitOverfittingUnderfitting

Typical training and validation loss patterns

Training loss

Validation loss

00.450.91.351.8123456789101112

Training loss continues falling while validation loss turns upward. The best checkpoint occurs before the final epoch.

Illustrative values, not measured results from the uploaded notebook.

Healthy fit

Both training and validation losses decrease and remain reasonably close.

This suggests that learning on training examples is transferring to validation data.

A small difference between the curves is not necessarily a problem.

Overfitting

Training loss continues decreasing while validation loss begins increasing.

The model is becoming more specialized to the training examples without improving generalization.

This is a typical reason to use regularization or early stopping.

Underfitting

Training and validation losses both remain relatively high.

Possible explanations include insufficient model capacity, incorrect optimization settings, too much regularization, or poor input representations.

It would generally be counterproductive to respond to underfitting by simply increasing dropout.

Why might validation loss rise while accuracy remains constant?

Suppose the correct class is A.

At one epoch, a misclassified example receives probabilities

$$ p(A)=0.4,\qquad p(B)=0.6. $$

The cross-entropy is

$$ -\log(0.4)\approx0.916. $$

At a later epoch, the network becomes more confident in the incorrect class:

$$ p(A)=0.01,\qquad p(B)=0.99. $$

Now the cross-entropy is

$$ -\log(0.01)\approx4.605. $$

The example was incorrectly classified in both cases, so its contribution to accuracy remains unchanged.

However, the loss becomes much larger.

Good student answer: How can validation loss increase without a decrease in validation accuracy?

Accuracy depends on which class has the highest predicted score. Cross-entropy also depends on confidence. A model that becomes more confident on incorrectly classified examples can have increasing validation loss even if the number of incorrect predictions stays the same.

Slide 27: Dropout disrupts feature co-adaptation

Dropout randomly disables some feature activations during training.

Suppose a hidden representation is

$$ \mathbf h= \begin{bmatrix} h_1\\h_2\\h_3\\h_4 \end{bmatrix}. $$

During training, dropout generates a binary mask

$$ m_i\sim\operatorname{Bernoulli}(1-p), $$

where p is the probability of dropping an activation.

The modified activations are

$$ \tilde h_i=\frac{m_i h_i}{1-p}. $$

This is called inverted dropout.

Why divide by $(1-p)$?

The expected value of the mask is

$$ E[m_i]=1-p. $$

Therefore,

$$E[\tilde h_i] = E\left[\frac{m_i h_i}{1-p}\right] = \frac{(1-p)h_i}{1-p} =h_i.$$

The scaling preserves the expected activation magnitude.

During evaluation, no mask is applied, and no additional scaling is required.

Numerical example

Let

$$\mathbf h= \begin{bmatrix} 2&4&6&8 \end{bmatrix}$$

and choose $p=0.5$.

One random mask might be

$$\mathbf m= \begin{bmatrix} 1&0&1&0 \end{bmatrix}.$$

Then

$$\tilde{\mathbf h} = \frac{\mathbf m\odot\mathbf h}{0.5} = \begin{bmatrix} 4&0&12&0 \end{bmatrix}.$$

On another training step, a different mask may be selected.

The network therefore cannot assume that every feature will always be available.

Why can this improve generalization?

Suppose the classifier has learned that one particular feature map is highly predictive of a siren.

Without dropout, it may become excessively dependent on that feature map.

With dropout, the classifier sometimes has to make predictions when that feature is absent.

This can encourage the use of multiple complementary cues.

However, dropout does not guarantee feature independence or eliminate shortcuts.

Good student answer: Why is dropout considered regularization?

Dropout introduces stochastic variation into the hidden representations during training. This makes the network less dependent on particular activations and can reduce overfitting. During evaluation, dropout is disabled so the complete learned representation is used.

Slide 28: Where should dropout go?

The lecture's baseline architecture places dropout between global pooling and the final linear classifier.

This is a sensible starting point because the global pooling layer produces a compact 64-dimensional representation.

Dropout acts on these summarized features rather than directly removing individual time-frequency measurements.

For example, if a 64-dimensional feature vector contains detectors corresponding to

- sustained harmonics,
- frequency modulation,
- broadband transients,
- low-frequency periodicity,

dropout may temporarily remove some of these cues during training.

This requires the linear classifier to become more robust to missing feature evidence.

Why not use extremely strong dropout?

If \(p=0.9\), approximately 90% of the activations are disabled on each training step.

For a 64-dimensional feature vector, only about 6 or 7 features survive on average.

That may leave too little information for reliable classification.

The model could underfit.

The slide suggests testing probabilities around 0.2–0.5, which are reasonable experimental candidates.

Standard dropout versus spatial dropout

Ordinary `nn.Dropout` independently masks individual elements of the tensor.

For convolutional feature maps, `nn.Dropout2d` can instead zero entire feature channels during training.

Channel-wise dropout may be more appropriate when neighboring spatial activations are strongly correlated.

Good student answer: Why place dropout after global pooling?

It regularizes the final class-level feature representation while preserving the local time-frequency structure used by earlier convolutional layers. It also makes the effect easier to interpret and tune than placing strong dropout throughout the feature extractor.

Slide 29: Small audio datasets demand fold discipline

The lecture uses two standard environmental sound datasets as examples.

Dataset	Recordings	Classes	Supplied folds
UrbanSound8K	8,732 excerpts	10	10
ESC-50	2,000 recordings	50	5

The key issue is not simply dataset size.

It is the number of statistically independent examples and the relationship between examples.

A dataset containing thousands of excerpts may provide much less independent information if many excerpts come from a small number of original recordings.

For meaningful validation, related fragments should remain grouped.

Example: Cross-validation with ESC-50

The dataset provides five predefined folds.

A simple teaching experiment could use:

- Folds 1–3 for training.
- Fold 4 for validation.
- Fold 5 for testing.

This is an illustrative nested holdout arrangement, not a substitute for the dataset's full official five-fold evaluation protocol.

For published comparisons, students should follow the prescribed evaluation procedure and report how hyperparameters were selected.

Good student answer: Why should supplied dataset folds be respected?

Predefined folds are designed to support comparable evaluations and often control relationships among recordings. Arbitrarily splitting excerpts can allow closely related recordings to appear in both training and validation, producing overly optimistic accuracy.

Part IV. SpecAugment: Training with incomplete spectrograms (Slides 30–35)

Slide 30: What is SpecAugment?

SpecAugment is a family of augmentation techniques that modify time-frequency representations during training.

In this lecture, the relevant operations are:

- Frequency masking
- Time masking

The idea is to remove selected portions of the spectrogram so the CNN cannot rely exclusively on one narrow region of acoustic evidence.

Unlike conventional image augmentation, these operations have specific physical interpretations.

A time mask removes information from a time interval.

A frequency mask removes information from a region of the frequency representation.

Original

Frequency mask

Time mask

Conceptual representation of an original log-Mel spectrogram and two types of masking. Black areas indicate deliberately removed feature information.

The intended result is not to create more data in a physical sense.

Rather, SpecAugment creates modified training examples that encourage the network to learn robust representations.

Slide 31: Frequency masking removes adjacent Mel channels

Frequency masking selects a contiguous range of Mel-frequency bands and replaces the corresponding values.

Suppose a spectrogram contains 64 Mel bands.

A mask might select bands 20 through 27.

All values in those bands are replaced across the selected time extent, which is typically the full duration of the spectrogram.

Mathematically, for a mask beginning at band f_0 with width W_f,

$$\tilde S[\ell,m] = \begin{cases} a, & f_0\leq\ell<f_0+W_f,\ S[\ell,m], & \text{otherwise}, \end{cases}$$

where a is the chosen mask value.

What is the physical meaning?

The model temporarily loses information about a frequency region.

This can simulate some consequences of missing frequency bands, spectral coloration, or narrowband interference.

It encourages the classifier to exploit other acoustic cues.

Example: Siren classification

Suppose a siren produces multiple harmonics.

A network that learns to depend almost entirely on one harmonic may be fragile.

Frequency masking sometimes removes that harmonic region during training.

The classifier must then use other harmonics, modulation patterns, or time-frequency structures.

When is frequency masking harmful?

Suppose a machine fault is characterized almost exclusively by a narrow spectral line at 7.2 kHz.

If frequency masking repeatedly removes the only discriminative band, the augmented training examples may no longer contain sufficient evidence for the label.

This can lead to underfitting or effectively invalid augmented examples.

Good student answer: Why does frequency masking sometimes improve generalization?

It discourages the classifier from depending on one narrow frequency region and encourages it to use redundant information across the spectrum. However, the mask width should be chosen carefully because removing the only diagnostic frequency component can make the classification problem ambiguous.

Slide 32: Time masking removes adjacent frames

Time masking selects consecutive time frames and replaces them with a mask value.

For a spectrogram $S[\ell,m]$, a time mask beginning at frame m_0 with width W_t can be written as

$$\tilde S[\ell,m] = \begin{cases} a, & m_0\leq m<m_0+W_t,\\ S[\ell,m], & \text{otherwise}. \end{cases}$$

Time masking can improve tolerance to temporary missing information, partial occlusion, or local variations in acoustic events.

Example: Spoken-word recognition

A spoken word contains several phonetic components.

If a short segment is masked, the network may still recognize the word from the remaining phonetic and spectral context.

The augmentation encourages the classifier to avoid relying exclusively on one short feature.

However, masking an entire short word or a uniquely identifying consonant may remove information essential to the label.

Good student answer

Time masking forces the model to classify sounds without access to every time frame. It can make the network more robust to partial temporal information loss, but excessive masking may eliminate the event being classified.

Slide 33: Where should SpecAugment occur?

The lecture shows the pipeline:

\[\text{Waveform} \rightarrow \text{Log-Mel} \rightarrow \text{Normalization} \rightarrow \text{Masking} \rightarrow \text{CNN}. \]

The significant point is that SpecAugment belongs in the training pipeline.

Validation and test transformations should be deterministic and should not apply random masks.

Why normalize before masking?

Suppose features are standardized to approximately zero mean.

Then replacing masked regions with zero corresponds roughly to replacing them with an average normalized feature value.

This avoids using an extremely large or small numerical value as a mask.

However, zero after normalization does not necessarily represent acoustic silence.

This distinction is important.

If the spectrogram is in dB before normalization, a value of 0 dB may represent the reference power level rather than silence.

Therefore, a zero-valued mask has different meanings depending on where it is inserted in the pipeline.

Good student answer: Why does the position of masking relative to normalization matter?

Because the numerical meaning of zero changes with the representation. After zero-mean normalization, zero generally represents an average feature value. Before normalization, zero in a dB spectrogram may represent the reference power rather than an absence of energy. Masking should use a fill value consistent with the feature representation.

Slide 34: Mask width has physical meaning

The width of a time mask can be converted to seconds.

Let

$$ W_t=\text{number of masked frames} $$

and

$$ R=\text{hop length in samples}. $$

Then the masked interval on the frame grid is approximately

$$ T_{\mathrm{mask}} = W_t\frac{R}{f_s}. $$

Example from the slide

Given

$$ W_t=20,\qquad R=160,\qquad f_s=16,000, $$

we obtain

$$ T_{\mathrm{mask}} = 20\frac{160}{16,000} = \boxed{0.20 \text{s}}. $$

The mask removes 200 ms of spectrogram frames.

This can be a small perturbation for a three-second clip containing a sustained signal.

But it can be a large perturbation for an event lasting only 100 ms.

Why is frequency-mask width harder to interpret?

A time-frame increment corresponds to a fixed hop interval.

A Mel-band increment does not correspond to a fixed number of hertz.

At low frequencies, neighboring Mel bands may be relatively close together in hertz.

At high frequencies, a similar number of Mel bands can cover a much wider frequency interval.

Therefore, masking eight Mel bands does not correspond to a fixed bandwidth in hertz.

Good student answer: How should SpecAugment mask widths be selected?

They should be chosen relative to the duration and frequency characteristics of the relevant acoustic events. Time-mask widths can be converted into seconds using the hop length and sample rate. Frequency-mask widths should be examined in the context of the actual Mel filter bank because the frequency scale is nonuniform.

Slide 35: Implementing SpecAugment in PyTorch

The slide presents a general-purpose masking function:

```
def mask_axis(x, axis, max_width):
    width = torch.randint(0, max_width + 1, ()).item()
    if width == 0: return x
    start = torch.randint(0, x.shape[axis] - width + 1, ()).item()
    index = [slice(None)] * x.ndim
    index[axis] = slice(start, start + width)
    x = x.clone()
    x[tuple(index)] = 0.0
    return x

x = mask_axis(x, -2, 8)   # frequency
x = mask_axis(x, -1, 16)   # time
```

Let's examine the main operations.

1. Choose a random mask width

```
width = torch.randint(0, max_width + 1, ()).item()
```

The mask width is sampled uniformly from zero through `max_width`, inclusive.

A width of zero means no masking occurs.

2. Choose a valid starting position

```
start = torch.randint(0, x.shape[axis] - width + 1, ()).item()
```

If the axis contains 64 elements and the mask width is 8, the starting position can range from 0 through 56.

This ensures that the mask remains inside the tensor.

3. Construct indexing slices

```
index = [slice(None)] * x.ndim
index[axis] = slice(start, start + width)
```

All axes are retained except the selected axis, where the mask interval is specified.

4. Apply the mask

```
x = x.clone()
x[tuple(index)] = 0.0
```

Cloning prevents the operation from unexpectedly modifying the original input tensor.

The selected region is then replaced with zeros.

5. Select the axis

For a spectrogram of shape `[1, M, T]`:

```
x = mask_axis(x, -2, 8)   # Frequency
x = mask_axis(x, -1, 16)  # Time
```

The `-2` axis is Mel frequency and `-1` is time.

This convention also works for a batched tensor `[B, 1, M, T]`, but masking the entire batch tensor this way applies the same mask location to every item. For per-example augmentation, apply it to individual examples or implement independent masks for each batch item.

One small improvement for robustness would be to restrict the mask width to the available axis length:

```
max_width = min(max_width, x.shape[axis])
```

Otherwise, requesting a mask wider than the tensor dimension will produce an invalid range when selecting the start position.

Good student answer: Why is `x.clone()` used?

It avoids modifying the original tensor in place. This is useful when the same features may be reused or when in-place changes could interfere with other computations. It also makes the augmentation function easier to reason about.

Part V. In-class activity and synthesis (Slides 36–37)

Slide 36: Predict, run, and debug

The activity asks students to trace an input tensor

```
[8, 1, 64, 173]
```

through the CNN, then investigate what happens if adaptive pooling is removed.

Question 1: Predict all tensor shapes

Assume three blocks, each consisting of a shape-preserving 3×3 convolution, batch normalization, ReLU, and 2×2 max pooling.

Stage	Tensor shape
Input	`[8, 1, 64, 173]`
Block 1	`[8, 16, 32, 86]`
Block 2	`[8, 32, 16, 43]`
Block 3	`[8, 64, 8, 21]`
Adaptive average pool	`[8, 64, 1, 1]`
Flatten	`[8, 64]`
Final linear layer (10 classes)	`[8, 10]`

Question 2: Use forward hooks to check shapes

PyTorch forward hooks can print the output shape of each layer.

```
model = AudioCNN(n_classes=10)
def print_shape(module, inputs, output):
    print(module.__class__.__name__, tuple(output.shape))
    handles = []
    for layer in model.features:
        handles.append(layer.register_forward_hook(print_shape))
    x = torch.randn(8, 1, 64, 173)
    with torch.inference_mode():
        output = model(x)
        for h in handles:
            h.remove()
    print("Final output:", output.shape)
```

This is a useful practical technique because students can compare their predicted dimensions with actual PyTorch behavior.

The number of trainable parameters can also be determined:

```
n_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print("Trainable parameters:", n_params)
```

For the three-block model defined above with ten classes, the parameter count is:

$$144+4608+18432+650+2(16+32+64)$$

$$=\boxed{24,058}.$$

The last term accounts for the trainable scale and shift parameters of the three batch-normalization layers.

BatchNorm running means and variances are stored buffers, not trainable parameters.

Question 3: What breaks without adaptive pooling?

Suppose we remove adaptive pooling and flatten the entire final feature map.

For time width $T=128$, three pooling operations produce

$$64\times8\times16.$$

Flattening gives

$$64(8)(16)=8192$$

features.

For $T=173$, the output is

$$64\times8\times21.$$

Flattening gives

$$64(8)(21)=10,752$$

features.

A linear layer constructed with

```
nn.Linear(8192, 10)
```

cannot accept an input containing 10,752 features.

The resulting error is a matrix dimension mismatch.

Good student answer:

Without adaptive pooling, the flattened feature-vector length depends on the spectrogram time dimension. The final linear layer requires a fixed input size, so a change in recording duration can cause a shape mismatch. Adaptive pooling maps every valid feature-map size to 64×1×1, ensuring that the linear layer always receives 64 inputs.

Part VI. Homework problems with expanded solutions (Slides 38–41)

Problem 1: Shape audit

Question: For input `[B,1,64,173]`, three 3×3 convolutions use padding 1 and stride 1, each followed by 2×2 max pooling. Find all spatial shapes.

Solution

The convolutions preserve spatial dimensions.

The pooling operations halve the dimensions, rounding down.

$$64\times173 \rightarrow32\times86 \rightarrow16\times43 \rightarrow8\times21.$$

The full feature shapes are:

$$[B,16,32,86]$$

$$[B,32,16,43]$$

$$[B,64,8,21].$$

With global adaptive pooling, the final output is

$$[B,64,1,1].$$

A good answer should include both the channel dimensions and the spatial dimensions.

Problem 2: Parameter count

Question: Count the trainable parameters in

```
nn.Conv2d(    16, 32, 3,    padding=1,    bias=False)
```

followed by

```
nn.BatchNorm2d(32)
```

Solution

The convolution has

\[32(16)(3)(3)=4608 \]

weights.

Since `bias=False`, it has no trainable bias vector.

Batch normalization has a trainable scale and shift parameter for each of the 32 channels:

\[32+32=64. \]

Total:

\[\boxed{4608+64=4672}. \]

A useful conceptual extension is to ask why batch normalization often allows the preceding convolution's bias to be omitted.

The reason is that the learned BatchNorm shift parameter can provide a channel-wise offset, making a separate convolution bias largely redundant in this configuration.

Problem 3: Variable time width

Question: Explain why flattening followed by a linear layer can fail for variable \(T\), and how adaptive pooling fixes it.

Solution

After three pooling stages, the feature map has dimensions approximately

\[[B,64,8,\lfloor T/8\rfloor]. \]

Flattening produces a feature vector of length

\[64(8)\lfloor T/8\rfloor. \]

Therefore, the number of inputs required by the linear layer changes with \(T\).

Adaptive average pooling produces

\[[B,64,1,1] \]

regardless of the input time width, provided the convolution/pooling stages produce valid output dimensions.

Flattening then always yields

\[[B,64]. \]

The classifier can therefore use

```
nn.Linear(64, n_classes)
```

for different spectrogram durations.

Problem 4: Cross-entropy loss debugging

Question: A model applies softmax and returns `[B,10]`, while the targets are floating-point tensors of shape `[B,10]`. Rewrite the intended standard single-label classification pipeline.

Solution

The model should return raw logits:

```
logits = model(x)
```

with

```
logits.shape == [B, 10]
```

For standard single-label classification, the targets should contain integer class indices:

```
y.shape == [B]y.dtype == torch.long
```

Then use

```
loss_fn = nn.CrossEntropyLoss()
loss = loss_fn(logits, y)
```

Two issues need to be distinguished.

First, applying softmax before cross-entropy is unnecessary and changes the intended loss calculation because the loss expects logits.

Second, float targets of shape `[B,10]` invoke PyTorch's probability-target mode rather than its integer-index mode. That mode is legitimate for correctly constructed soft-label targets, but it is not what the lecture intends.

A strong answer should explain why both choices matter rather than simply saying that float targets are always invalid.

Problem 5: Diagnose the learning curves

Question: Training loss decreases from 1.8 to 0.05. Validation loss reaches a minimum at epoch 9 and then increases to 1.4. Propose a controlled three-run experiment.

Suggested answer

The pattern is consistent with overfitting.

First, establish that validation is deterministic and the data split is correct.

Then perform three training runs under controlled conditions:

Run	Model configuration	Purpose
A	Baseline CNN	Establish reference learning curves
B	Baseline + dropout	Measure the effect of dropout
C	Baseline + SpecAugment	Measure the effect of input masking

Keep the train-validation split, initialization seed policy, optimizer, learning-rate schedule, epoch budget, and other preprocessing choices consistent.

For each run, compare the lowest validation loss, the corresponding epoch, validation accuracy or class-sensitive metrics, and the generalization gap.

The expected behavior, if regularization is effective, is that training loss may decrease more slowly while validation performance improves.

This is not guaranteed. SpecAugment might be harmful if the masks erase the only relevant event.

An additional fourth run could combine dropout and SpecAugment after their separate effects have been measured.

Good student answer: Why shouldn't we introduce dropout, SpecAugment, and stronger weight decay simultaneously?

Because changing several regularization mechanisms at once makes it difficult to identify which one caused an improvement or degradation. Controlled experiments should isolate the effect of each intervention before testing combinations.

Problem 6: SpecAugment and ESC-50 folds

Question: At 16 kHz with a hop of 160 samples, calculate the duration of a 24-frame mask. Design training and validation transformations for an ESC-50 fold.

Solution

The frame interval is

$$ \Delta t=\frac{160}{16,000}=0.01 \text{s}. $$

A 24-frame mask corresponds to

$$ T_{\mathrm{mask}}=24(0.01)=\boxed{0.24 \text{s}}. $$

A reasonable training pipeline is:

\[\text{Audio} \rightarrow \text{Resample} \rightarrow \text{Random crop} \rightarrow \text{Log-Mel} \rightarrow \text{Normalize} \rightarrow \text{SpecAugment}. \]

The validation pipeline is:

\[\text{Audio} \rightarrow \text{Resample} \rightarrow \text{Deterministic crop or fixed windows} \rightarrow \text{Log-Mel} \rightarrow \text{Normalize}. \]

Both use the same basic DSP feature parameters and the same normalization statistics estimated from the training data.

Only the training pipeline applies random masking.

The supplied dataset folds must be respected.

A particularly strong answer would mention that ESC-50 recordings are five seconds long, so any shorter patch policy needs to justify whether the label remains valid for the selected segment.

Part VII. Weekly quiz — Good answers (Slides 42–45)

The deck includes eight short quiz questions. These are appropriate for checking basic conceptual understanding after the lecture.

Question	Good answer
1. What two axes does Conv2d traverse?	The Mel-frequency axis and the time-frame axis.
2. What does `AdaptiveAvgPool2d((1, 1))` produce from `[B,64,H,W]`?	`[B,64,1,1]`.
3. Should softmax be applied before `CrossEntropyLoss`?	No. Pass raw logits. The loss function performs a numerically stable log-softmax calculation internally.
4. Name the five operations in a training step.	Zero gradients, forward pass, calculate loss, backward pass, update parameters.
5. What changes when `model.eval()` is called for dropout?	Dropout stops randomly masking activations. All features are active without additional scaling.
6. Training loss falls while validation loss rises. What is the pattern?	It is consistent with overfitting. Possible interventions include dropout, weight decay, augmentation, a smaller model, or early stopping after verifying the validation protocol.
7. At 16 kHz and hop	\(10(160)/16,000=0.10\) seconds.

Question	Good answer
160, how long is a 10-frame mask?	
8. Why respect supplied UrbanSound8K or ESC-50 folds?	They help control dependencies among recordings and enable comparable evaluations using established dataset protocols.

For grading, I would distinguish between a correct short answer and an explanatory answer.

For example, "overfitting" is sufficient to identify the pattern in Question 6, but a stronger response explains that the network continues to improve on the training set while its performance on independent validation recordings deteriorates.

Part VIII. Three additional conceptual questions I would ask

These questions go somewhat beyond the slides but connect the material to the students' DSP background.

Question A: Is a CNN convolutional layer really a linear system?

Consider

```
nn.Conv2d(1, 16, kernel_size=3, padding=1)
```

followed by

```
nn.ReLU()
```

Suggested answer

The convolution operation is linear with respect to its input when its weights are fixed and the bias is zero.

If a bias is included, the layer is affine rather than strictly linear.

The ReLU operation is nonlinear:

$$\operatorname{ReLU}(x)=\max(0,x).$$

For example,

$$\operatorname{ReLU}(-1)=0,$$

but

$$-\operatorname{ReLU}(1)=-1.$$

Thus, ReLU does not satisfy the scaling property of a linear system.

A CNN combines linear filtering with nonlinear transformations, allowing it to represent functions that cannot be implemented by a single linear filter.

This is a useful connection between the students' earlier LTI-system coursework and their newer machine learning material.

Question B: Why is global average pooling not equivalent to calculating the average input spectrogram?

Suggested answer

Global average pooling occurs after multiple convolutional and nonlinear layers.

The pooled quantities are learned feature activations, not the original spectrogram values.

For example, one feature channel might respond strongly to chirps, while another responds to impulses.

Averaging these feature maps summarizes the presence or average strength of the corresponding learned patterns.

Simply averaging the original spectrogram would instead produce a much less expressive global energy summary.

The distinction is that nonlinear feature extraction occurs before pooling.

Question C: Can dropout or SpecAugment correct a contaminated training-validation split?

Suggested answer

No.

Regularization modifies the model's learning behavior, but it does not restore statistical independence between training and validation recordings.

If related recordings appear in both subsets, the measured validation score can remain biased even when dropout and SpecAugment are used.

The data split and evaluation protocol must first be corrected.

Part IX. Recommended teaching emphasis for the 75-minute lecture

The slide deck divides the class into four sections. I would preserve that structure and use the following questions to connect the sections.

Time	Material	Key question students should answer
0–25 min	Audio CNNs	What does each convolution or pooling operation do to the time-frequency information?

Time	Material	Key question students should answer
25–45 min	Training loop	How does cross-entropy become parameter updates through backpropagation?
45–60 min	Overfitting	What evidence distinguishes overfitting, underfitting, and evaluation errors?
60–75 min	SpecAugment	Which parts of a spectrogram can be masked without invalidating the label?

The three most important takeaways

1. CNN architecture encodes assumptions about relevant signal information.

A convolution kernel's dimensions, the amount of pooling, and the use of global average pooling determine what local features can be learned and what positional information is retained.

The architecture should follow from the acoustic classification problem, rather than being selected solely because it works well on conventional images.

2. A correct training loop is necessary but not sufficient.

The correct order of gradient operations, use of logits with cross-entropy, appropriate optimizer settings, and correct evaluation mode are essential implementation details.

However, a technically correct training loop can still learn the wrong relationships if the data pipeline, labels, or split are flawed.

3. Regularization should be selected in response to observed failures.

Dropout reduces dependence on particular internal feature activations.

Weight decay discourages excessive parameter magnitudes.

SpecAugment reduces dependence on particular time-frequency regions.

Early stopping selects a checkpoint before validation performance deteriorates.

These mechanisms address different aspects of model behavior and should be compared through controlled experiments.

Suggested culminating discussion question

> We train a CNN on three-second log-Mel spectrograms to classify environmental sounds. The model achieves 99% training accuracy and 65% validation accuracy. We add dropout and SpecAugment, and validation accuracy improves to 75%. Can we conclude that the new model is better?

A strong student response would be:

The increase in validation accuracy is encouraging, but additional evidence is needed. We should verify that both models were evaluated on the same independent validation split using deterministic transformations, and that the improvement is reproducible across seeds or folds. We should inspect class-specific performance, ensure no group leakage exists, and reserve an independent test set for final evaluation. We should also check that masking does not invalidate the labels. Only then can we confidently attribute the improvement to better generalization.

That question brings together everything in the lecture: representation, architecture, optimization, regularization, experimental design, and evaluation.

The strongest overall connection to the DSP portion of the course is that the CNN is learning nonlinear combinations of time-frequency filtering operations, while the engineer is still responsible for ensuring that those operations preserve the physically meaningful information needed for the task.